

Test Case Prioritization Using Extended Digraphs

SEYEDEH SEPIDEH EMAM and JAMES MILLER, University of Alberta

Although many test case prioritization techniques exist, their performance is far from perfect. Hence, we propose a new fault-based test case prioritization technique to promote fault-revealing test cases in *model-based testing* (MBT) procedures. We seek to improve the fault detection rate—a measure of how fast a test suite is able to detect faults during testing—in scenarios such as regression testing. We propose an extended digraph model as the basis of this new technique. The model is realized using a novel *reinforcement-learning* (RL)- and hidden-Markov-model (HMM)-based technique which is able to prioritize test cases for regression testing objectives. We present a method to initialize and train an HMM based upon RL concepts applied to an application's digraph model. The model prioritizes test cases based upon forward probabilities, a new test case prioritization approach. In addition, we also propose an alternative approach to prioritizing test cases according to the amount of change they cause in applications. To evaluate the effectiveness of the proposed techniques, we perform experiments on *graphical user interface* (GUI)-based applications and compare the results with state-of-the-art test case prioritization approaches. The experimental results show that the proposed technique is able to detect faults early within test runs.

CCS Concepts: **Software and its engineering** → **Software testing and debugging** • Software and its engineering → *Formal software verification*

Additional Key Words and Phrases: Fault-based test case prioritization, model-based testing (MBT), reinforcement learning, additional statement coverage, HMM, GUI testing, random prioritization

ACM Reference Format:

Seyedeh Sepideh Emam and James Miller. 2015. Test case prioritization using extended digraphs. ACM Trans. Softw. Eng. Methodol. 25, 1, Article 6 (November 2015), 41 pages.
DOI: <http://dx.doi.org/10.1145/2789209>

1. INTRODUCTION

The complexity and size of software systems are growing, along with the increasing importance of testing and verifying these systems. As a result, many test suites produced during development are reused in a regression testing mode, especially during software maintenance or evolution. Decreasing regression testing costs while increasing fault detection power is an important goal in software testing; this challenge can potentially be addressed by *model-based testing* (MBT) techniques [El-Far and Whittaker 2002; Legeard and Utting 2006]. MBT has two phases: (1) the generation of executable test cases; and (2) the execution and evaluation of test cases [Hemmati 2011]. The focus of this study is only on the second phase to evaluate the generated test cases and prioritize them based upon suggested factors. Researchers have proposed various methods to reduce the second-phase costs. These techniques can be divided into three significant categories: *test suite minimization techniques* (which select a minimal subset of test suites with respect to maintaining the original coverage) [Farooq 2011; Parsa and Khalilian 2010; Jeffrey and Gupta 2007], *test case selection techniques* (which identify

Author's addresses: S. S. Emam and J. Miller (corresponding author), University of Alberta, 116 St. & 85 Ave., Edmonton, AB T6G 2R3, Canada; email: jimm@ualberta.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2015 ACM 1049-331X/2015/11-ART6 \$15.00

DOI: <http://dx.doi.org/10.1145/2789209>

the test cases relevant to the set of recent modifications) [Frankl et al. 2003], and *test case prioritization techniques* (which sort the test cases by determining their execution priority based upon different criteria) [Elbaum et al. 2002; Rothermel et al. 2001; Zhang et al. 2009a]. Despite the existence of safe test selection and minimization techniques [Rothermel and Hall 1997; Rothermel and Harrold 1998], empirical evidence shows that some of them can severely decrease the fault detection capabilities of test suites, or disturb the conditions under which safety can be achieved [Rothermel et al. 1998]. For instance, Hemmati et al. [2013] evaluated several similarity-based selection techniques (STCS) and then compared the effectiveness of the best similarity-based selection technique with other common selection techniques in the literature. The results show that the best STCS is never worse than non-STCS techniques, regardless of the failure rate. On the other hand, test case prioritization techniques schedule test cases in order to increase their capabilities to address significant issues [Rothermel et al. 2001], specifically:

- increasing the rate of fault detection;
- increasing the code coverage;
- increasing the confidence of testers in the reliability of the *system under test* (SUT);
- increasing the rate of detecting high-risk defects; and
- increasing early detection of faults which are correlated with specific code changes.

Running all of the test cases and then fixing the faults may lead to the execution of unnecessary test cases containing redundant functionality and delay in the regression testing process. Within this domain of application, the following reasons motivate us to propose a novel prioritization technique.

- We can reduce the time required to execute test cases and increase the likelihood of spending testing time more beneficially in the case of an unexpected termination of regression testing activities.
- During the test case prioritization process, no test case is discarded. Hence, it can be concluded that the prioritization techniques do not suffer the drawbacks that can occur when the test case selection and test suite minimization mechanism discards test cases.
- In the situation that decreasing the number of test cases is considered as an objective, prioritization techniques can be applied in conjunction with minimization and selection methods. So test case prioritization can be applied as a complementary step in regression testing.
- It is also worth noting that, in regression testing, we may be interested in prioritizing test cases in a way that can be effective for a particular version. Elbaum et al. [2002] call this approach “version-specific prioritization”. In this study, we also focus on version-specific test case prioritization.

Therefore, in this article, after a comprehensive explanation about the proposed techniques and their computation and estimation processes, we walk through applying them on a successful model-based technique in *Graphical User Interface* (GUI) testing [Mariani et al. 2012], while later we show that it can be used with any type of MBT technique. In addition, the domain of application is also not restricted to GUI testing; it is believed that the approach is appropriate to most software testing scenarios. Finally, we compare the approach with traditional (random, additional statement coverage, worst, and optimal) prioritization schemes, as well as with a recent contribution in the literature, to demonstrate its effectiveness. Specifically, the principle contributions of this article are as follows.

- We introduce a new behavioral model, the extended (multi-) digraph. This model provides a richer set of information than traditional behavioral models such as a regular digraph. It is argued that this extended model offers superior performance.
- We give a concrete mechanism to (numerically) populate this extended model. Specifically, it shows how to use reinforcement learning and hidden Markov models to realize this model.
- We demonstrate how this model can be utilized to automatically prioritize test cases for GUI applications.
- We empirically show that the technique(s) derived from this extended model outperform other common dynamic test case prioritization techniques.

In the next section, we present the motivations behind this research. In Section 3, we provide some required background information about reinforcement learning and hidden Markov models. Section 4 represents several test case prioritization techniques that have been used in this research. Sections 5 and 6 include the design of the proposed techniques by describing the RL-based HMMs' parameters estimation and prioritization methods, and a short motivating example. Section 7 describes the evaluation phase and experimental setup. It also discusses the results and analysis of our empirical studies. Section 8 provides a discussion on the achieved results. Section 9 discusses some work related to the contribution of this research. Section 10 considers the threats to internal and external validity of the study, and, finally, Section 11 presents overall conclusions and some thoughts on potential future work.

2. MOTIVATION

The concept of test case prioritization is a well-established task in software verification and validation. Several different test case prioritization techniques with various aims have been considered through multiple research experiences. The most common aim is increasing the fault detection rate of a test suite; this is also the main motivation of this article. The prioritization technique can be *dynamic*, building upon dynamic dataflow information, or *static*, based upon static analysis of the *System Under Test* (SUT). This section presents the motivations behind proposing a new dynamic prioritization technique.

2.1. Static or Dynamic Prioritization

Most prioritization techniques are based upon structural code coverage information gathered through dynamic execution of the SUT, while others focus on static analysis of the test cases. Static prioritization techniques mainly focus on ordering the test cases based on analyzing the source code or test cases' static call graph. For example, Mei et al. [2012] propose a static prioritization approach to estimating the ability of each test case to achieve code coverage, and using this information for reordering the test cases. Prioritizing test cases based on static approaches suffers from the following drawbacks [Yoo and Harman 2007].

- Since this technique walks through the source code, there is a need to have access to the SUT or the test cases' source code.
- Choosing an appropriate method to statically analyze the source code depends on the programming language used to write the code. Thomas et al. [2012] present a static technique ranking test cases by extracting and analyzing topic models. Their approach is only applicable in Java-based and medium-sized SUTs, which also indicates that the generalizability of this approach is not certain.
- Prioritizing test cases using their static call graph also affects both the prioritization precision and cost. Since a static call graph intends to represent every possible execution of the program, the problem of generating an exact static call graph is

undecidable. So generating a comprehensive and correct static call graph causes overapproximations, or unrealistic transitions and relationships.

These reasons, along with the proven capability of dynamic approaches in effectively ranking test cases, imply that developing new dynamic test case prioritization techniques is a viable alternative.

2.2. Overview of the Utilized Testing Models and Domain of Application

Our dynamic test case prioritization uses an MBT-based approach. It assumes that an MBT-based *front-end* (automatic test case generator) exists and, in essence, “extends” this front-end model and system to complete the testing process. In MBT, the test modeler creates an abstract model (state machine) of the SUT and then generates a set of test cases by walking (traversing) through the model [Dev et al. 2012].

So, MBT is about the automatic generation of efficient tests using models of the SUT [Arlt et al. 2012]. This model is a depiction of SUT behavior, including input sequences (actions, conditions, output logic, and dataflow) through modules and routines [Hemmati 2011]. The most significant feature of MBT is automating both the test generation and execution processes along with the capability of generating test cases. For instance, Lindlar et al. [2010] combine MBT with *Evolutionary Functional Testing* (EFT) to achieve a fully automatic test case design and evaluation framework. Meanwhile, Endo and Simao [2011] represent a new approach to apply model-based testing in service-oriented applications’ testing process.

The behavioral model is often instantiated as labeled multi-digraphs where nodes are connected to each other via multiple directed edges. The model is built and updated according to an agent’s activities [Mariani et al. 2012]. This approach has been successfully used in generating GUI behavioral models using many different techniques to automatically generate test suites that simulate user behavior in interacting with GUIs. In particular, model-based GUI testing techniques generate test cases based upon a state transition diagram. Several techniques with diverse modeling methods have been proposed [Memon 2007; Mariani et al. 2011; Reza et al. 2007]. Most of them depict a graph-based model where nodes represent events relating to GUI widgets, and edges (or arcs) represent the relationship between events.

Hence, a GUI test case can be generated from such a model by selecting any possible path. In this study, our proposed technique is able to prioritize generated test cases using any such MBT approach. Our approach is based upon building an extended multi-digraph as a SUT behavioral model. In the following, definitions of both regular and extended digraphs are considered. Extended digraphs are the basis for our research contribution.

Definition 2.1. We define a regular digraph as $G = (S, E, V)$ consisting of a set of nodes (S) representing GUI states of the SUT, and edges ($e_k \in E = (s_i, s_j)$, where $s_i, s_j \in S$) indicating the transition between states. The transitions are labeled with the name of the actions triggering transitions and, in some cases, an estimated utility value of the action.

Figure 1 shows an overview of a regular digraph. In this figure, a_{ij} denotes the utility of the executed action between states s_i and s_j (obviously, transitions between s_j and s_i and between s_i and s_k , where $k \in 1, 2 \dots N$ and $k \neq i$, are also feasible). The method of calculating this value is explained in Section 5.

Definition 2.2. An extended digraph $G' = (S, E, V, A, B)$ is an extended version of a regular digraph G which contains additional information. In the extended digraph, V indicates a set of actions triggering transitions, where $v_k \in V (k \in 1, 2 \dots M$ and M is the number of possible actions), A is the probability transition matrix in which

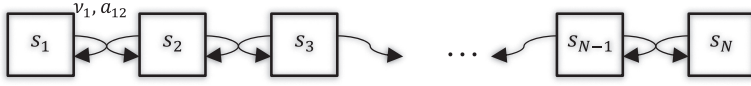


Fig. 1. An overview of the behavioral model as a directed graph.

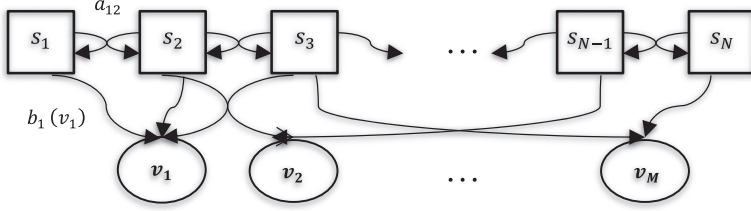


Fig. 2. An overview of the behavioral model as an *extended* directed graph.

each element $a_{ij} \in A$ represents the probability of moving from state s_i to state s_j , and B shows the emission matrix in which each element $b_i(v_k) \in B$ indicates the probability of observing action v_k in state s_i (Figure 2). The methods of estimating and calculating these elements are also considered in Section 5. The extra information helps in evaluating the effect of executing actions on GUI states, enabling superior prioritization of test cases.

Generating the extended model only requires simple interfacing glue code to be added to pre-existing MBT approaches. So the extended model can be generated using information gathered during the test case execution procedure, hence there is no need to re-implement test cases to generate the model. Once generated, this model can be utilized to prioritize the test cases (to maximize their effectiveness during regression testing and other activities). To elaborate the motivation behind this approach, we consider the following example.

The main reason for proposing a stochastic approach is the proven ability of such techniques in estimating these probabilistic models, specifically in calculating the forward probability of a sequence of events; in this study, the sequence of events is interpreted as a test case. It should be noticed that the proposed techniques in this research are not limited to GUI-based applications and can be applied in a wide range of circumstances. However, to demonstrate the utility of the approach, a domain of application needs to be selected. We have chosen to evaluate our techniques by prioritizing test cases of GUI-based applications for the following reason.

- GUIs are very common, and the need for GUI testing tools becomes ever greater.
- The number of generated test cases in GUI testing is often large. Hence it is time consuming to reexecute them during regression testing [Memon et al. 2008].
- No GUI test case generation tool prioritizes test cases for regression testing objectives.
- We also choose to do so because event-flow graphs [Memon 2007], action-based behavioral models [Mariani et al. 2012], or other types of SUT models contain information about user-observations (based on interacting with the SUT). Hence, it is feasible to determine the observable and latent states required to estimate the extended behavioral model.

Thus, based upon a well-known GUI testing definition [Memon 2007], the focus of this research would be on prioritizing test cases consisting of a sequence of events (an episode), which is performed during user-GUI interactions. (An episode is a path from the initial to a terminal state, or a sequence of produced executions by an agent.)

In other words, we only cover testing interactions that are performed between a user and the GUI; other types of test cases are outside our scope. The definition is further restricted as actual users would introduce variation in experimental results, hence, in our experiments, we utilize an automated test case generation tool which acts as a proxy for actual users [Mariani et al. 2012]. This allows for exploration of a fully automated testing process from user action generation to test case prioritization.

3. THEORETICAL BACKGROUND

The proposed prioritization technique is established by combining *Reinforcement Learning* (RL) and *Hidden Markov Model* (HMM) concepts to efficiently and rapidly prioritize test cases. The main reasons for choosing Reinforcement Learning are its strong statistical background, its proven ability in handling a wide range of data, and its ability to re-estimate the Markov model efficiently. Using RL, we are able to estimate an appropriate HMM and then use it to compute each test case's forward probability, that is, the likelihood of executing a specific test case based upon the SUT's inferred HMM. Briefly, this method proposes a probabilistic-based prioritization technique using the following steps:

- estimating initial RL-based HMM parameters according to the generated test cases using MBT techniques;
- training an RL-based HMM with a maximum likelihood using the Baum-Welch algorithm [Baum et al. 1970]; and
- prioritizing test cases based on estimating the forward probability of each test case in the execution phase by applying a forward algorithm [Rabiner and Juang 1986].

The RL-based HMM is estimated using the generated model through MBT techniques. This approach uses a System Under Test (SUT) behavioral model (currently an extended digraph graph; however, changing the model is straightforward) to generate a Markov chain containing hidden states and transition probabilities estimated using RL algorithms. Thus, this technique is able to prioritize test cases based upon the amount of computations (changes) that a test case may cause in GUI states (using Q-learning, a type of RL) and the probability of each action happening in each specific state (using the HMM).

Moreover, we investigate another prioritization approach (called accumulated Q-value) by computing the accumulated amounts of Q-values for each specific test case. This can be interpreted as a prioritization technique where the technique is derived from only a test case's contribution in changing an application's state. It is worth noting that the Q-values can be calculated during generation of the extended behavioral model based on Definition 2.2. The method of this computation will be considered in Section 4. Before investigating the proposed techniques, some background on RL and HMM is provided.

3.1. Reinforcement Learning

Reinforcement Learning (RL) contains one of the best-known classes of machine learning algorithms which “teach” an agent how to interact with an environment [Wiering and van Otterlo 2012]. It is located between supervised and unsupervised learning since only a limited feedback, named a *reward signal*, is received by an agent about the agent's predictions [Szepesvári 2010]. The long-term objective of this agent is performing an action (i.e., mapping situations to actions) which maximizes the overall reward signal [Sutton and Barto 1998]. RL's practical applications not only address learning paradigms in operations research and control engineering [Rabiner and Juang 1986; Chang 2006; Davoodi and Andrews 1998; Arel et al. 2010], but are also one of the most active research areas in artificial intelligence [McPartland and Gallagher 2011]. RL

algorithms' progress is typically iterative. They learn during iterations by observing the current environment, inferring the environment's state, and executing an action which guides the agent to the next state. In other words, the agent receives the system's state and the reward score associated with the last transition. Then, it evaluates potential actions according to the expected reward to be realized, and selects an action which is sent back to the system. In response, the system makes a transition to a new state, and this cycle will be repeated as part of a *Markov Decision Process* (MDP) [Wiering and Otterlo 2012; Szepesvári 2010]. MDPs can be categorized as stochastic extensions of finite automata or as Markovian processes which are augmented by actions and rewards, so they consist of actions and transitions as well as states.

Definition 3.1. The system under consideration is called Markovian if executing an action does not depend on previous actions and visited states (i.e., it only depends on the current state and status). So, an MDP contains:

- a finite set of environmental states $S = \{s_1, s_2, \dots, s_N\}$, where N is the number of states;
- a finite set of actions $A = \{a_1, a_2, \dots, a_k\}$, where k is the size of the action space;
- the transition function $T : S \times A \times S \rightarrow [0, 1]$ which computes the probability of reaching the state s' by performing action a in state s , and is denoted as $(T(s, a, s'))$; and
- to compare different states and actions, during agent and environment interaction, they must be ordered according to their occurrence, so s_t denotes the state at time t [Wiering and Otterlo 2012].

Thus, according to the definition of a Markovian process, we have

$$P(s_{t+1}|s_t, s_{t-1}, s_{t-2}, \dots) = P(s_{t+1}|s_t) = T(s_t, a_t, s_{t+1}). \quad (1)$$

Definition 3.2. The *reward function* R specifies the reward, or penalty, the agent receives by performing an action. So, $R : S \times A \times S \rightarrow \mathbb{R}$ presents the reward function that computes the immediate utility of an action to define the model of the MDP. So an MDP can be denoted by the tuple $\langle S, A, T, R \rangle$ depicting it as a state transition graph [Wiering and Otterlo 2012].

Definition 3.3. The value function $V^\pi(s)$, specifies “how good” it is for the agent to be in a given state. The *how good* notation here is expressed in terms of future rewards that can be expected. We can define the value of state s under policy π , formally $V^\pi(s)$, as [Sutton and Barto 1998]

$$V^\pi(s) = E_\pi \{R_t | s_t = s\} = E_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \middle| s_t = s \right\}, \quad (2)$$

where the stochastic policy $\pi : S \times A \rightarrow [0, 1]$ is a mapping from each state s and action a to the probability $\pi(s, a)$ by performing an action a when in state s where E_π is the expected value earned by following policy π and *discount factor* γ , with $0 \leq \gamma < 1$. This models the fact that future rewards are worth less than an immediate reward, that is, if $\gamma = 0$ the agent only would be concerned about the immediate reward, while a value close to 1 gives a large weighting to future actions.

Definition 3.4. Similarly, the value of performing an action a in state s (the state-action value function, or Q-value function, $Q : S \times A \rightarrow \mathbb{R}$) can be defined as

$$Q^\pi(s, a) = E_\pi \{R_t | s_t = s, a_t = a\} = E_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \middle| s_t = s, a_t = a \right\}. \quad (3)$$

Almost all RL-based paradigms are based on estimating the value functions appropriately, which has led to the exploration and production of several different estimating methods and techniques. One of the most popular, which also is applied in this research, is *Q-learning* [Watkin 1989].

Q-learning is a method to estimate Q-value functions when there is no available model of the MDPs (*model-free* fashion). In this situation, because of the lack of a priori transition and reward models, there is a need for sampling and exploration to learn the required model, or to step directly into estimating values for actions (Q-values). Therefore, Q-learning estimates the agent's Q-value function based upon an action's Q-value estimation; this process is incrementally evaluated as follows [Wiering and van Otterlo 2012]:

$$\mathcal{Q}_{k+1}(s_t, a_t) = \mathcal{Q}_k(s_t, a_t) + \alpha \left(r_t + \gamma \max_a \mathcal{Q}_k(s_t, a) - \mathcal{Q}_k(s_t, a_t) \right), \quad (4)$$

where α ($0 < \alpha \leq 1$) is the *learning rate*, which determines the extent to which new information can override old [Dayan and Watkin 1992]. Because of its proof of convergence to an optimal policy [Lin 1992] and proven ability in value function estimation in free-model problems [Watkin 1989], we apply Q-learning for estimating the Q-values in this research.

3.2. Hidden Markov Model

Hidden Markov Models are popular statistical tools for modeling data in various areas such as bioinformatics, speech recognition, partial discharges, and many other temporal pattern situations [Yin and Wang 2001; Lee et al. 1990; Aas and Eikvil 1996]. In the broadest sense of the word, an HMM is a Markovian process that is split into two components: the observable states and the hidden (latent) states. Compared to an HMM, regular Markov models only consist of observable states which are directly visible to the observer. Therefore only the state transition probabilities require estimation. However, in a hidden Markov model, only the outputs (observations), depending on the latent states (GUI states), are visible. Therefore, as the GUI states are invisible from an observer view and only the outputs in this model are completely observable, regular Markov models and *Partially Observable Markov Decision Processes* (POMDPs) are not applicable in this study [Williams and Young 2007]. An HMM is usually characterized by the following elements [Ching 2006]:

- N, the number of hidden states in the model, $S = \{s_1, s_2, \dots, s_N\}$;
- M, the number of distinct observation symbols per hidden state, $V = \{v_1, v_2, \dots, v_M\}$;
- the state transition probability distribution $[A]_{ij} = \{a_{ij}\}$, where $a_{ij} = P(Q_{t+1} = s_j | Q_t = s_i)$, $1 \leq i, j \leq N$;
- the observation symbol probability distribution in hidden state j, $[B]_{jk} = \{b_{j(v_k)}\}$, where $b_{j(v_k)} = P(O_t = v_k | Q_t = s_j)$, $1 \leq j \leq N$, $1 \leq k \leq M$; and
- the initial state distribution $\Pi = \{\pi_i\}$, where $\pi_i = P(Q_1 = s_i)$, $1 \leq i \leq N$.

Using the values of N, M, A, B, and Π , the HMM can be used as a generator to create an observation sequence (where T is the number of observations in the sequence): $O = \{O_1, O_2, O_3, \dots, O_T\}$. We use the notation $\Lambda = (A, B, \Pi)$ to simply indicate the complete parameter set of the HMM. There are three key issues with HMMs which are commonly considered when applying them to a problem domain.

Problem 1. Given the observation sequence $O = \{O_1, O_2, O_3, \dots, O_T\}$ and an HMM, how do we efficiently compute the probability of the observation sequence?

Problem 2. Given the observation sequence $O = \{O_1, O_2, O_3, \dots, O_T\}$ and an HMM, how do we find the state sequence that best explains the observations?

Problem 3. Given the observation sequence $O = \{O_1, O_2, O_3, \dots, O_T\}$, how do we choose the model parameters in an HMM?

In this research, we present a novel application of HMMs in software testing, that is, we show how they can be utilized in test case prioritization processes by addressing the first and third issues. The third one can be solved in software testing by estimating the SUT's hidden Markov model (based on the software's behavioral model) and extracting the known sequence of test cases (observations). Having the model and observation sequence, we will be able to accurately compute the forward probability of each test case using a forward dynamic programming procedure. This probability is what we need to evaluate and prioritize test cases; this evaluation also resolves the first issue of applying HMMs to this new problem domain. Addressing the second problem is beyond the scope of this initial research; however, a suggested approach is briefly presented in Section 11.

3.2.1. Forward Probability. A forward algorithm computes the probability of encountering a sequence of observations, supposing that the sequence has been generated by a given HMM. In another words, a forward algorithm is used to calculate a belief state or the probability of a state at a certain time (forward probability). A forward probability can be calculated at each time step by considering the most likely state, given the previous history. Therefore, the forward probability $\alpha_k(t)$ is the sum of the probabilities of all constrained paths of length i (where i is the length of a sequence of observations) that end in state k .

Since test suites are considered as a set of observed sequences, this algorithm estimates the forward probability of each test case, given an HMM learned from an extended digraph.

According to the RL approach (Section 5) that has been used in this study, the forward probability of a test case is related to the amount of computation (changes) in GUI states; test cases with higher amounts of changes are more likely to encounter unexpected behavior than a test case causing fewer changes. In other words, test cases that activate more actions during their execution process are more likely to detect faults. Thus, a forward algorithm is an appropriate choice to compute this likelihood, as a set of test cases can be modeled as a sequence of actions. This procedure has the following steps [Rabiner and Juang 1986].

—We define the forward probability, $\alpha_k(t)$, as the joint probability of observing the first t vectors v_t , $T = 1, \dots, t$ while in state k at time t . Another way to state this would be that $\alpha_k(t)$ is the probability of observing $v_1, v_2, \dots, v_t$, while, in addition, at time t the state is k .

$$\alpha_k(t) = P(v_1, v_2, \dots, v_t, s_t = k | \Lambda). \quad (5)$$

—This probability can be evaluated by the following recursive formula:

$$\begin{aligned} \alpha_k(1) &= \pi_k b_k(v_1), \quad 1 \leq k \leq N \\ \alpha_k(t) &= \pi_k b_k(v_t) \sum_{l=1}^N \alpha_l(t-1) a_{l,k}, \quad 1 \leq k \leq N, 1 \leq t \leq T \\ \alpha_k(t) &= \sum_{l=1}^N \alpha_l(t-1) b_k(v_t) a_{l,k}. \end{aligned} \quad (6)$$

However, when the sequences of observations (the length of the episodes) become larger, the probabilistic values in the forward algorithm get increasingly small and, after multiple iterations, the values tend to zero. For that reason, $\alpha_k(t)$ are scaled during the iterations of the algorithm to avoid underflow problems. The scaling coefficients are used to keep the probability values in the dynamic range of the machine. So, the coefficient c_t is defined as follows [Blunsom 2004]:

$$c_t = \frac{1}{\sum_{k=1}^N \alpha_k(t)}. \quad (7)$$

Using c_t , the scaled value of $\alpha_k(t)$ would be

$$\alpha_k^*(t) = c_t \times \alpha_k(t) = \frac{\alpha_k(t)}{\sum_{k=1}^N \alpha_k(t)}. \quad (8)$$

To conclude, after estimating an appropriate HMM, we would be able to prioritize test cases by computing their corresponding forward algorithm.

4. TEST CASE PRIORITIZATION

The *test case prioritization* (TCP) approach sorts test cases within a test suite in order to maximize some predefined criteria such as additional code coverage or fault detection rate [Wong et al. 1997]. In other words, test cases with the highest score with respect to the prioritization criteria have the highest priority to be executed. Rothermel et al. [2001] provide a formal definition for the test case prioritization problem.

Definition 4.1. Given: T (a test suite), PT (a set of permutations of T), and f (a function that maps PT onto a real number). Problem: Find $T' \in PT$ such that

$$(\forall T'') (T'' \in PT) (T'' \neq T') [f(T') \geq f(T'')]. \quad (9)$$

In this definition, PT is the set of all possible orderings of test suite T . In addition, function f yields an award value for each specific prioritization showing its value (prioritizations with highest award values are preferable). We customize this definition based upon the research's objectives in the following section.

Several different test case prioritization techniques have been proposed through multiple research experiences. Elbaum et al. [2002] consider 18 different approaches and compared them in terms of effectiveness with respect to cost and performance. Moreover, they classify them into three separate groups. The first group is named *comparator*, containing random and optimal ordering. The second one is the *statement-level* group, consisting of four fine-granularity techniques. The third group is the *function-level* group, which contains 12 coarse-granularity techniques. In this research, we will consider random, optimal, and worst ordering techniques to define upper and lower boundaries in achieving the test case prioritization goal (here, improving the fault detection rate). In addition, we prioritize test cases based upon the additional statement coverage technique from the second category, and two novel RL-based prioritization approaches, which will be illustrated in following sections. Since the number of statements is greater than the number of functions and the statement-level group of techniques is a "superset" of the function-level techniques, we don't consider the function-level group and just apply the statement coverage techniques.

4.1. Random, Optimal and Worst Prioritization Techniques

One of three prioritization techniques which we consider in this research is random test case prioritization. In this approach, we randomly pick a test case among the list of test cases and remove it from the list, then we repeat the picking process until no test case exists [Zhang et al. 2009b]. We consider it as an experimental control to

evaluate the effectiveness of other heuristics in comparison with the random ordering. We also select and consider the optimal and worst prioritization approaches. Both of these techniques are not practical, since information about which test cases reveal faults and which do not expose them is not available [Elbaum et al. 2002]. But in this research the SUT's faults are known and we can determine their corresponding test sets. Thus, we consider optimal and worst prioritization techniques to theoretically obtain the upper and lower boundaries in the fault detection rate. In the optimal prioritization, we prioritize test cases in an order which maximizes this rate, while in the worst prioritization approach, we are trying to prioritize test cases that can detect the fewest new faults, thus minimizing the fault detection rate [Huang et al. 2010].

4.2. Additional Statement Coverage Prioritization

Determining which statements in the program were explored (covered) by a specific test case can be considered through a program instrumentation phase. Then, test cases can be prioritized according to the total number of statements they cover (*total coverage prioritization*) [Rothermel et al. 2001]. It is possible to achieve additional coverage in subsequent testing processes by considering statements that have not been covered in earlier processes. So, here we use additional statement coverage prioritization which is based upon feedback about the coverage gained so far.

We iteratively select a test case that covers the maximum number of statements, then we adjust the coverage information of the remaining test cases to calculate the coverage of statements that have not been covered [Elbaum et al. 2002]. This process is repeated until all of the statements are covered by at least one test case. It is worth noting that, if the multiple test cases cover the same number of not previously covered statements, we need to choose one of them in a random way. Nardo et al. [2013] demonstrate that the additional coverage approach is the best coverage-based prioritization technique in terms of fault detection capability. They evaluated the effectiveness of different coverage-based prioritization techniques on an industrial case study and concluded that prioritization methods based on additional coverage using finer-grained coverage criteria outperformed all other coverage-based techniques.

Cobertura¹, an open-source Java tool, is able to determine which parts of a Java program are covered by a test case; Cobertura can be embedded into a test case generator (AutoBlackTest, in this article). AutoBlackTest collects coverage data incrementally during the execution of test cases on the current version of the application. Gained information is utilized in order to prioritize test cases in an additional statement coverage manner.

5. TEST CASE PRIORITIZATION USING RL-BASED HMM

Combining hidden Markov model and reinforcement learning approaches as a part of a stochastic estimation process has been considered in several artificial intelligence problems. Hamamoto et al. [2004] suggest using RL to re-estimate the HMM parameters when the recognition model does not work during motion prediction scenarios. Ho et al. [2002] also integrate HMMs and RL to recognize communication channels and translate human actions into instruction symbols. This process was done by estimating HMMs and then applying RL to decide the next fixation point. On the contrary, Kabudian et al. [2004] propose an RL-based approach to train HMMs in order to cope with the local optima problem that happens when using a local search method.

In this research, we propose a novel approach to prioritize test cases using an RL-based HMM. In other words, we want to learn a model which is not only able to generate test cases by traversing model paths but also can easily rank test cases based upon

¹<http://cobertura.github.io/cobertura>.

their forward probabilities. Achieving this goal is possible by integrating the HMM and RL approaches, producing a graph-based model as an MDP-based learning technique. The RL-based HMM approach provides a three-step framework: the first step contains a reinforcement learning algorithm, here Q-learning, to learn to interact with the software under test, to stimulate its functionalities, and to automatically generate test cases. The second step includes an HMM estimation process to prioritize generated test cases. The final step computes the test cases' forward probabilities using the *forward algorithm*. Before going through the prioritization steps, we state two assumptions which are used to realize our work in the GUI testing domain.

- In this domain, actions represent GUI actions such as click, select, deselect, etc. According to Mariani et al.'s [2012] research, Q-values indicate the values of actions based upon the computations activated by them. They believe that the reward function should encourage the system to perform actions triggering “large volumes of computations” (*critical events* [Huang et al 2010]) rather than those that only activate “small volumes of computations”. For instance, clicking on a button or typing a word in the text area causes smaller changes to the GUI state than filling out a form and submitting it to a database. So the actions that contribute more GUI changes are more likely to gain higher rewards, and consequently higher Q-values, than others. Also, based upon this research, a GUI state consists of a set of widgets $\{w_1, \dots, w_n\}$ and each widget (w_i) can be interpreted as a pair $(type_i, P_i)$, where $type_i$ indicates the type of a widget such as textarea, label ... and P_i is a collection of widget properties and their values. We accept and build upon Mariani et al.'s [2012] work in this respect. They also define a function that generates traits from widgets, where a trait is subset of the widget properties. So, given w_1, w_2 ; $w_1 =_t w_2$ iff $trait(w_1) = trait(w_2)$. For example, the trait of a button includes its type, position, and label shown on it. A similar approach is also offered by IBM functional Tester² to compare widgets.
- A GUI visualizes and processes data through menus, buttons, labels, etc. Extracting such information embedded into widgets can be helpful to automatically produce interesting executions in the application. Becce et al. [2012] suggest an algorithm to extract the descriptions of widgets. Mariani et al. [2012] use this technique in designing their GUI testing framework. Again, we accept and build upon this work [Becce et al. 2012; Mariani et al. 2012].

5.1. Step 1: Q-Learning Estimation Method

As mentioned earlier, Q-learning is one of the most popular Q-value function estimation techniques in the situation where there is no access to a predefined MDP model. Algorithm 1 shows the Q-value estimation process customized for GUI testing [Mariani et al. 2012]. It computes the reward function based upon the proportion of properties that change value, and then the fraction of widgets that are different when observed in different states. According to the computed reward function and initial inputs, we are able to estimate the Q-values for each specific episode.

5.2. Step 2: HMM Parameter Estimation

As mentioned before, a hidden Markov model has five significant elements; each element requires an initial estimation. For the HMM $\Lambda = (A, B, \Pi)$ with a (discrete) observation distribution, we consider the following parameters.

- N . This is number of hidden states, which is equivalent to the total number of states (possible widgets such as save or open) in the SUT.

²<http://www-03.ibm.com/software/products/us/en/functional>.

- M . This is the number of distinct observation symbols, which is equivalent to the number of possible events or actions (such as click, double click, or typing something in a text box).
- The state transition probability ($N \times N$) matrix is the probability of traversing from one state to an adjacent state. In RL, and in particular Q-learning, we are able to assign an RL score or Q-value to each edge of the generated model. Because this value is assigned after execution of the corresponding action to the edge, it represents the likelihood of the corresponding transition occurring [Mariani et al. 2012]. It can be computed (Algorithm 1) for each s during action a 's execution: $Q(s, a) = \text{reward}(s, a) + \gamma \max_{a'} Q(\delta(s, a), a')$. If more than one possible action between two adjacent states exists, the action with the largest Q-value would be selected as the value of the transition probability between the two connected (hidden) states.
- The observation probability matrix is an $N \times M$ matrix (emission matrix) calculated as follows, where f_{jk} is the frequency of observing a specific action (v_k) in a specific state j :

$$b_j(v_k) = \begin{cases} \frac{f_{jk}}{\sum_{k \in v} f_{jk}}, & f_{jk} > 0 \\ 0, & f_{jk} = 0 \end{cases} \quad (10)$$

- The initial state probability distribution, describes the starting state of the model. To generate the model in this study, every state is considered as a potential starting state.

ALGORITHM 1: Customized Q-Learning for GUI-based Applications

Input n , number of executions that the agent must produce for each episode (length of episode)

Initialize $Q(s, a) \leftarrow 0, \forall s \in S, \forall a \in A$ [Mariani et al. 2012]

$\gamma \leftarrow 0.9$

$i \leftarrow 0$

$\varepsilon \leftarrow 0.8$

For each episode **do**

s is a random state initialized as the starting state

Repeat

- Choose a possible action $a \in A(s)$
- Perform action a based on ε - *greedy*** policy [Lin 1993]
- $i \leftarrow i + 1$
- Observe the new state s'
- Compute the $\text{reward}(s, a)$:
 - Compute diff_w : the degree of change to a GUI widget while moving from s to s' :
 $\text{diff}_w(w_1, w_2) \leftarrow \frac{|P_1 \setminus P_2| + |P_2 \setminus P_1|}{|P_1| + |P_2|}$, where $w_1 =_t w_2$, are the same widgets monitored in s and s'
 - Compute $\text{diff}_s(s, s') \frac{|s \setminus s'| + \sum_{w_1 \in s, w_2 \in s', w_1 =_t w_2} \text{diff}_w(w_1, w_2)}{|s'|}$, where $|s|$ denotes the fraction of widgets in state s .
 - $\text{reward}(s, a) \text{diff}_s(s, \delta(s, a))$, δ denotes the state reached by executing action a .
- Compute $Q(s, a) \leftarrow \text{reward}(s, a) + \gamma \max_{a'} Q(\delta(s, a), a')$
- $s \leftarrow s'$

Until n is equal to $(i-1)$.

* $\leftarrow$ shows the assignment

** ε - *greedy* policy selects a random action with probability ε and the one with highest Q-value with probability $1 - \varepsilon$

Now, we have an initial estimation for each required element of the HMM with, respectively, N and M latent and observable states. This model illustrates two types of relationships.

- An edge label between latent states i and j represents the (a_{ij}) transition probability between these two states.
- An edge label between latent state j and visible state k ($b_j(v_k)$) is calculated using the preceding formula.

In the next step, this initial model is used as an input to an *expectation-maximization* (EM) algorithm; specifically we utilize the Baum-Welch algorithm [Baum et al. 1970]. This algorithm estimates the best model as that with the highest likelihood of the estimated parameters solving the first problem (see Section 3.2). If we consider each test case as a sequence of observations, we are able to predict the forward probability of test cases by utilizing the estimated HMM and implementing the forward algorithm.

5.3. Step 3: Computing Forward Probabilities and Considering their Application in Test Case Prioritization

Before applying the forward probability in prioritizing test cases, the reasons for choosing it as a prioritization metric should be reviewed. According to the forward probability calculation procedure (see formulas (5) and (6) in Section 3.2.1), both transition and emission probabilities are essential parameters in computing the forward probability $\alpha_k(t)$. In addition, as mentioned earlier (see steps 1 and 2, and Algorithm 1), the transition probabilities are directly calculated using the amount of Q-values (Q-values indicate the likelihood of the corresponding transition occurring by computing the amount of changes triggered during the transition). Therefore, it can be concluded that the RL-based HMM prioritizes test cases based on the amount of changes that can be triggered in the GUI state (Q-values) by executing each specific sequence of observations (actions). This technique not only considers the state changes, but also evaluates the effect of a specific action's performance on each state (HMM emission matrix). So both the states and edges of the application's behavioral model are involved in the prioritization technique. According to Huang et al. [2010], test cases containing more critical events (large volumes of computations) are more likely to uncover faults during testing. Therefore, test cases with higher forward probabilities are more likely to contain such events than those with lower forward probability.

Now, in order to complete the procedure of applying forward probabilities in test case prioritization, we modify Rothermel et al.'s [2001] definition to involve the forward algorithm in calculating an award value (Definition 4.1). In the new definition, F_p would be a new function which yields forward probabilities for each "test case". It is worthwhile to mention that, in the original definition, we compute the award value for each different ordering, but in the new definition, we calculate it for every test case in an offline mode with no need to re-execute the test cases. Therefore, we are able to sort test cases by assigning higher ranks to those with higher amounts of forward probabilities.

Definition 5.1. Given: T (a test case), TC (a set of generated test cases), and F_p (a function that maps from TC to a real number). Problem: Find $T' \in TC$ such that

$$(\forall T'')(T'' \in TC)(T'' \neq T')[F_p(T') \geq F_p(T'')]. \quad (11)$$

According to the new definition, we prioritize test cases based upon the estimated HMM. The following algorithm describes the proposed method. It accepts a set of test cases $TC = \{T_1, T_2, \dots, T_c\}$, where $T = \{v_1, v_2, \dots, v_t\}$ is a sequence of events or actions. Then, it estimates a corresponding HMM and prioritizes the test cases according to their F_p value. It is worth noting that, if the multiple test cases have the same amount of F_p s (which rarely happens), we need to choose one of them in a random way.

ALGORITHM 2: RL-based HMM Test case Prioritization Algorithm**Input:** $TC = \{T_1, T_2, \dots, T_c\}$; **Output:** TC' : Prioritized TC**1: While** $\exists T_i \in TC$ **do:** **For each** T_i **do**

- $TC' \leftarrow T_i$ and Extract all corresponding hidden states (S_i): $N \leftarrow N + 1$ (per visit to each unique state)
- Extract all corresponding actions (v_i): $M \leftarrow M + 1$ (per visit to each unique event)
- If a_{ij} does not exist then, compute Q-value among S_i and S_j and assign it as a_{ij} into the transition matrix $[A]_{N \times N}$
- If b_{ij} does not exist then, compute the corresponding observation symbol probabilities and assign it as b_{ij} into matrix $[B]_{N \times M}$

2: Train HMM (Λ) by implementing Baum-Welch (A, B, Π)**3: While** $\exists T_i \in TC'$ **do:**

- $F_{pi} = forward(\Lambda, T_i) = \alpha_k(t) = P(v_1, v_2, \dots, v_t, s_t = k | \Lambda)$

4: $TC' \leftarrow DescendSort(TC')$ based upon F_{pi} values**6. ACCUMULATED TEST CASES' Q-VALUES IN DESCENDING ORDER**

In order to investigate the effect of test case dissimilarities on fault detection rate, we also decided to prioritize test cases based upon the amount of computations activated by the corresponding action in each test case. As mentioned earlier, GUI state changes can easily be calculated using the Q-learning technique. Given each test case's accumulated Q-value (summation of each test case's Q value), we only need to rank every test case in a descending order, then label the one with the highest accumulated Q-value (SQ) as the test case with the highest execution priority. Algorithm 3 illustrates the overview of this technique.

ALGORITHM 3: Accumulated Test Cases' Q-values Ordering**Input:** $TC = \{T_1, T_2, \dots, T_c\}$ **Output:** TC' : Prioritized TC**1: While** $\exists T_i \in TC$ **do:** **For each** T_i **do**

- $TC' \leftarrow T_i$
- Extract all corresponding Q-values ($Qvalue_j$)
- $SQ_i = \sum(Qvalue_j)$

2. $TC' \leftarrow DescendSort(TC')$ based upon SQ_i values

This technique can be considered as an attempt to optimally use the information in a regular digraph, whereas the RL-HMM technique explicitly utilizes additional information only available in the extended diagram. In the following section, we discuss how these techniques improve the test case prioritization problem with regard to the failure detection objective.

6.1. Motivating Example

In this study we cover two well-known MBT-based GUI testing tools: GUITAR³ and AutoBlackTest⁴ (ABT). GUITAR uses a test automation framework based on state machines, named the event-flow model [Memon 2007], while AutoBlackTest generates a behavioral model based upon agent activity [Mariani et al. 2011].

³http://sourceforge.net/apps/mediawiki/guitar/index.php?title=GUITARHome_Page.

⁴<http://www.lta.disco.unimib.it/tools/AutoBlackTest>.

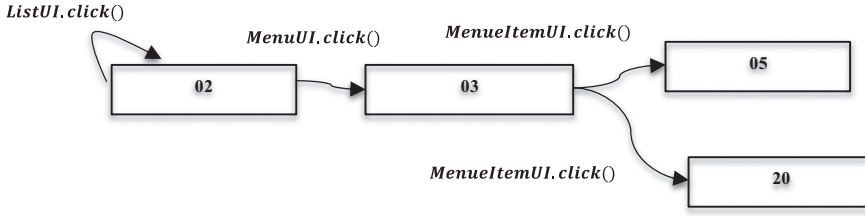


Fig. 3. Excerpt of a behavioral model as a directed graph motivating example.

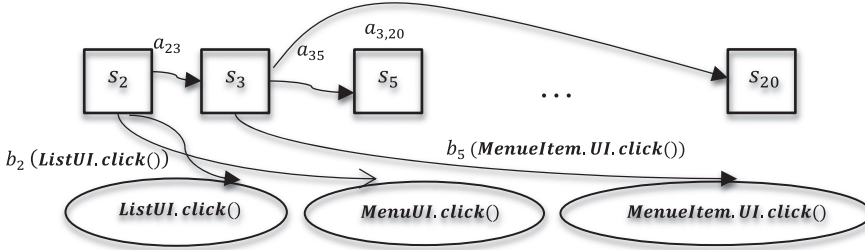


Fig. 4. Excerpt of a behavioral model as an extended directed graph motivating example.

To motivate this work, we consider a small example to investigate the difference between state-of-the-art behavioral models (such as a regular digraph) and the extended multi-digraph. Figure 3 shows a small excerpt of a behavioral model derived by a GUI-based MBT test case generation tool (AutoBlackTest) for the UPM application (an application for building a personal database of accounts and one of the case studies presented in this article) [Mariani et al. 2012]. In this figure, the nodes of the graph represent the GUI states of the SUT and the edges represent the transition between the states. They are labeled with the name of the actions triggering the transitions and, in some cases, the estimated utility value of the action. This is an example of a regular digraph. Applying our new technique, we would be able to generate an extended form of directed graphs utilizing stochastic information. The additional information will be used in the test case prioritization procedure to calculate the forward probability of each test case. Figure 4 depicts an excerpt of an extended behavioral model applying our new approach on the UPM application.

Based on Definition 2.2, this figure is an extended behavioral model consisting of a set of GUI states (rectangular shapes), actions (elliptical shapes), and additional information (labels). In this model a_{ij} indicates the transition probability from state i to j and $b_i(v_k)$ indicates the probability of observing action v_k in state i . This is the only information necessary to store for test case prioritization, specifically, if it is stored in the form of two probability matrices covering a_{ij} and $b_i(v_k)$ for all possible transitions.

Thus, considering following sequences as possible test cases extracted from the prior directed graph (Figure 3), we would be able to calculate required data to estimate an RL-based HMM and prioritize test cases based upon the forward probabilities. This obviously shows that the extended directed graph contains more useful information than the regular one, which helps in estimating the probabilistic model and calculating the forward probability of a sequence of events (i.e., test cases). As mentioned earlier, the forward probability of the sequence is used to effectively prioritize test cases by assigning a higher fault detection probability to test cases which are able to trigger

more actions and computations in each GUI state.

$$\begin{aligned} T_1 &= \{02(ListUI.click()), 02\}. \\ T_2 &= \{02(MenuUI.click()), 03\}. \\ T_3 &= \{02(MenuUI.click()), 03(MenuItemUI.click()), 05\}. \\ T_4 &= \{02(MenuUI.click()), 03(MenuItemUI.click()), 20\}. \end{aligned}$$

For example, for Test Case 2 (T_2), we should calculate the Q-value and the immediate utility value of an action using the following reward function, based on the differences between widgets in each state (Algorithm 1). So

$$\begin{aligned} reward(s, a) &= diff_s(s, \delta(s, a)) = diff_s(02, \delta(02, MenuUI.click())) \\ &= \frac{|s \setminus_t s'| + \sum_{w_1 \in s, w_2 \in s', w_1 =_t w_2} diff_w(w_1, w_2)}{|s'|} = 0.55. \end{aligned}$$

In addition, because this test case terminates in State 3 and no more actions are executable from this state, $\gamma max_{a'} Q(\delta(s, a), a') = 0$ and

$$\begin{aligned} a_{23} &= Q(s, a) = reward(s, a) + \gamma max_{a'} Q(\delta(s, a), a') = 0.55, \\ b_2((MenuUI.click())) &= \frac{f_{jk}}{\sum_{k \in v} f_{jk}} = \frac{1}{2} = 0.5. \end{aligned}$$

Calculating the same parameters for all test cases gives the following transition probability and emission matrices.

$$[A] = \begin{bmatrix} 0 & 0.55 & 0 & 0 \\ 0 & 0 & 0.71 & 0.7 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad [B] = \begin{bmatrix} 0.5 & 0.5 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0. \end{bmatrix}.$$

Having all the required parameters to estimate an accurate HMM prepares the model for RL-HMM-based prioritization. Estimating the RL-based HMM using the Baum-Welch algorithm, we would be able to compute the forward probability of each test case using the forward algorithm. Therefore, applying formula (6), we would have

$$\begin{aligned} (Forward(RL - HMM, T1)) &= 0.137, \\ (Forward(RL - HMM, T2)) &= 0.275, \\ (Forward(RL - HMM, T3)) &= 0.5, \\ (Forward(RL - HMM, T4)) &= 0.5. \end{aligned}$$

And, we can sort test cases in following order:

- (1) T_4, T_3, T_2, T_1 ,
- (2) T_3, T_4, T_2, T_1 .

As we expected, test cases with larger quantities of forward probabilities should be prioritized due to their higher amounts of computations and actions which have been triggered in the GUI states during their execution. We expect that test cases with such characteristics are able to reveal more faults in GUI-based applications because of their capabilities in triggering more actions in GUI states. T_4 matches with Episode 2 in the complete example (Section 7.3.1), which is able to reveal a fault in the *Application Under Test* (AUT). So, choosing either the first or second ordered test suite, we would be able to detect a fault by running only one or two test cases.

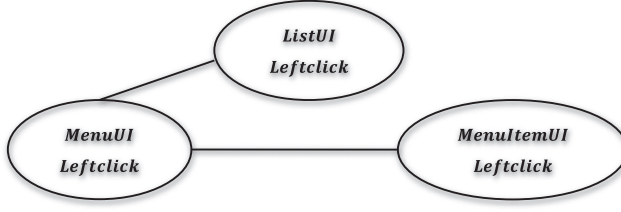


Fig. 5. Event-flow graph extracted from GUITAR (motivating example).

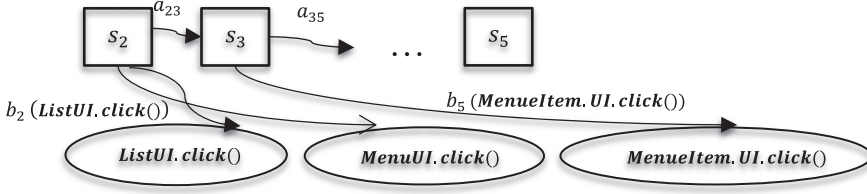


Fig. 6. Extended directed graph generated using event-flow graph.

Also, we can go further in this example and replace the initial model, which is generated by Mariani et al.'s [2012] technique, with the event-flow graph generated using GUITAR [Memon 2007], another GUI test case generator tool (Figure 5). This procedure shows that the proposed test case prioritization technique is applicable to any test cases that have been generated by any GUI-based MBT test case generation tool.

Unlike the graph, which was extracted from AutoBlackTest, GUITAR's event-flow graph is not a state-based model and only contains the relations between actions. Based upon the changes between GUI states during the test case generation process [Bece et al. 2012], we are able to create a GUITAR-based extended digraph (Figure 6). In this situation, we assume that GUI state has been changed at least one time in each event execution.

Based upon the generated event-flow graph, GUITAR is able to generate the following test cases:

$$\begin{aligned} T_1 &= \{(ListUI.click())\}; \\ T_2 &= \{(ListUI.click()), (MenuUI.click())\}; \\ T_3 &= \{(ListUI.click()), (MenuUI.click()), (MenuItemUI.click()),\}. \end{aligned}$$

Similar to the previous steps, both emission and transition probability matrices should be calculated before estimating the hidden Markov model. Similarly, the amount of Q-values and immediate utility values should be computed based upon the differences between the widgets in corresponding GUI states. Matrix A indicates the transition probabilities and B shows the emission probabilities in the initial HMM.

$$[A] = \begin{bmatrix} 0 & 0.55 & 0 \\ 0 & 0 & 0.71 \\ 0 & 0 & 0 \end{bmatrix}, \quad [B] = \begin{bmatrix} 0.5 & 0.5 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

Calculating the forward probabilities, we would have the following results.

$$\begin{aligned} (\text{Forward}(RL - HMM, T1)) &= 0.25; \\ (\text{Forward}(RL - HMM, T2)) &= 0.068; \\ (\text{Forward}(RL - HMM, T3)) &= 0.5. \end{aligned}$$

Subsequently, we can sort the test cases in the following order: T_3, T_1, T_2 .

The test suite generated by GUITAR missed the fault-revealing test case T_4 during the test case generation phase; thus, in this case, the final test suite is not able to detect any fault and prioritization doesn't affect the speed of defect detection. This demonstrates the higher ability of AutoBlackTest in generating test cases with better coverage. In addition, it highlights the capability of our proposed approach in prioritizing test cases that have been generated using different GUI test case generators. Our approach is clearly independent of their architectures and strategies.

7. EMPIRICAL EVALUATION

To investigate the effectiveness of the proposed techniques (RL-based HMM and accumulated Q-value) relative to existing prioritization techniques, we have constructed an experimental framework and addressed the following research questions.

- (RQ1). How does the RL-based HMM technique compare in terms of effectiveness with other prioritization techniques?
- (RQ2). How does the accumulated Q-value technique compare in terms of effectiveness with other prioritization techniques?
- (RQ3). Does RL-based HMM prioritize test cases more effectively than the accumulated Q-value approach?
- (RQ4). Does RL-based HMM prioritize test cases more effectively than the weight-based methods presented by Huang et al. [2010]?

7.1. Comparison Criteria

7.1.1. Average Percentage Faults Detected (APFD). Based upon the test case prioritization goal, there are many possible comparison criteria that can be utilized in order to evaluate the effectiveness of the applied approach [Rothermel et al. 2001]. In this research, we have focused on the goal of increasing the “likelihood of revealing faults earlier in the execution of the test run”, or, in the other words, “the fault detection rate” during regression testing.

To measure how rapidly a prioritized test suite is able to detect faults, we have used the APFD measure. APFD is defined as [Rothermel et al. 2001]:

$$APFD(T') = 1 - \frac{TF_1 + TF_2 + \dots + TF_m}{nm} + \frac{1}{2n}. \quad (12)$$

where T' is a prioritized test suite, T a test suite containing n test cases, and F a set of m faults revealed by T . In addition, TF_i is the first test case in the ordering T' of T that detects fault i . The range of APFD is between 0 and 1, and a higher APFD means superior fault detection.

7.1.2. Statistical Testing⁵. Arcuri and Briand [2011] provide guidance on the statistical analysis of automated processes in software engineering contexts. We essentially follow their advice and present effect-size estimations [Ellis 2010] as the principle descriptor of performance differences between algorithms. The estimation of effect size is

⁵We have implemented the statistical results and plotted the boxplots using OriginLab.

commonly preceded by traditional statistical testing, and hence we provide both for the sake of completeness.

The central limit theorem states [Galon 1889] that the distribution of the mean of a sufficiently large number of independent, identically distributed variables will be approximately normal, regardless of the underlying distribution. In addition, Rice [1994] and many other texts provide a rule of thumb saying that ~ 30 data points are sufficient to get reliable results. Hence, a solid theoretical basis exists for applying a paired t-test on the mean of these types of variables. Therefore, we “normalized” our statistical analysis by executing the prioritization techniques 1,000 times for each of the generated test suites and construct the mean value of the APFD (robust estimator). Then, for each prioritization approach, we select a prioritized test suite (R) with minimal discrepancy between the mean APFD. Thus, we have the following.

- In this study, the random variable is the mean of the percentage of faults detected by a given prioritization technique over all orderings.
- X_i, Y_i indicate two paired measurements from the n measured values; the measured values are the mean of the percentage of faults detected by a given prioritization technique (i) on a test suite of size (n).

Therefore, after computing this (ordered) element, we would be able to apply statistical testing by calculating the differences between the mean of the percentages of detected faults in all braces of paired sets X_i and Y_i (paired test). To apply the paired t-test, the t-statistic needs be calculated as shown.

$$\begin{aligned}\hat{X}_i &= (X_i - \bar{X}) \\ \hat{Y}_i &= (Y_i - \bar{Y}), \\ t &= (\bar{X} - \bar{Y}) \sqrt{\frac{n(n-1)}{\sum_{i=1}^n (\hat{X}_i - \hat{Y}_i)^2}}\end{aligned}\tag{13}$$

This brace-wise approach is standard practice in such situations in many domains [Hsu & Lachenbruch 2008] when the variable meets the paired t-test assumptions⁶.

In this case, we established a null hypothesis and an alternative hypothesis to be evaluated. The null hypothesis (H_0) states that the two prioritization techniques provide the same effectiveness of fault detection if the mean of the percentage of detected faults (over all orderings) for both techniques is the same. On the other hand, alternative hypothesis (H_1) states that, if the difference between the mean of the percentages of faults which have been detected by each of the prioritization techniques is not zero, then they will be considered as different. We independently evaluate this hypothesis for a number of random situations; the randomization is proved by changing the SUT under consideration without pattern or rationale. Therefore, by considering a significance level $\alpha = 0.05$, we would be able to reject the null hypothesis if $p\text{-value} < 0.05$ for each independent situation.

7.1.3. Effect Size. In order to add a “size of difference” statement to our comparison criteria and normalize the statistical evaluation of multiple (with independent subjects (SUT)) tests, we principally consider the strength or magnitude of a treatment effect by calculating Cohen’s d measure. Cohen’s d is defined as the difference between the means, $M_1 - M_2$, divided by *standard deviation* (SD), s_{pooled} . So,

$$d = \frac{M_1 - M_2}{s_{pooled}},\tag{14}$$

⁶As a cross-check, the samples for this test were subjected to a Shapiro-Wilk test for normality. In all situations, the samples passed the test.

where

$$s_{pooled} = \sqrt{SD_{group1}^2 + SD_{group2}^2 / 2}.$$

Since Cohen's d computes the standardized difference between two groups, or a brace, of samples, it can help to interpret the strength of our proposed techniques in comparison with others. Cohen [1988] suggests that $d = 0.2$ can be considered as a small effect size, while 0.5 can be interpreted as a medium effect size, and 0.8 large effect size. In this research, we consider this measure on the percentage of detected faults which have been detected by a brace of prioritization techniques with mean APFD, in order to investigate the magnitude of each approach's effect. So, for each brace:

$(R_i, R_j), i, j \in \{All\ considered\ prioritization\ approaches\}$, both $\overline{P}_i$ and $\overline{P}_j$ (the average of percentage of detected faults) should be calculated to compute:

$$d = \frac{\overline{P}_i - \overline{P}_j}{s_{pooled}}, s_{pooled} = \sqrt{SD_{R(i)}^2 + SD_{R(j)}^2 / 2}. \quad (15)$$

7.2. Experimental Setup

According to a recent empirical comparison, AutoBlackTest outperforms GUITAR in both code coverage and the number of detected faults [Mariani et al. 2012]. Therefore, in this study, we principally use AutoBlackTest to generate the required test cases.

7.2.1. AutoBlackTest and Modified Framework. AutoBlackTest is an automatic testing tool that builds a model of a SUT and produces test cases by walking through the model. The most significant feature of AutoBlackTest is its ability to interact with an unknown environment. To address this issue, AutoBlackTest uses a Q-learning approach [Mariani et al. 2012].

Therefore, AutoBlackTest is a combination of a Q-learning agent and a test case selector. The Q-learning agent is responsible for executing a sequence of actions (episodes) extracted from the model. On the other hand, the test case selector eliminates redundant test cases which cover the same statements within the code.

The derived model by AutoBlackTest is a regular digraph [Mariani et al. 2012]. Therefore, according to Definition 2.1, AutoBlackTest generates a graph $G = (S, E, V)$, where the set of nodes (S) represents the abstract GUI states and the edges ($e_k \in E = (s_i, s_j)$), where $s_i, s_j \in S$ indicate the transition between states. The transitions are labeled with the name of actions and corresponding Q-values (V).

It is believed that AutoBlackTest is a state-of-the-art mechanism to automatically infer a regular digraph, an essential building block for construction of the extended digraph. While AutoBlackTest is a state-of-the-art technique, it is far from perfect and only constructs a portion of the total regular digraph for the "average" GUI-based application. Figure 7 depicts the proposed framework and procedure to determine effectiveness of the proposed approaches, both in terms of fault detection and in comparison with other existing techniques (random, optimal, worst, coverage-based, and accumulated Q-value). The experimental environment consists of two steps.

Step1. Examine the generated test cases by parsing episodes produced by AutoBlackTest. Each episode, or test case, is a subgraph of the SUT's behavioral model, which

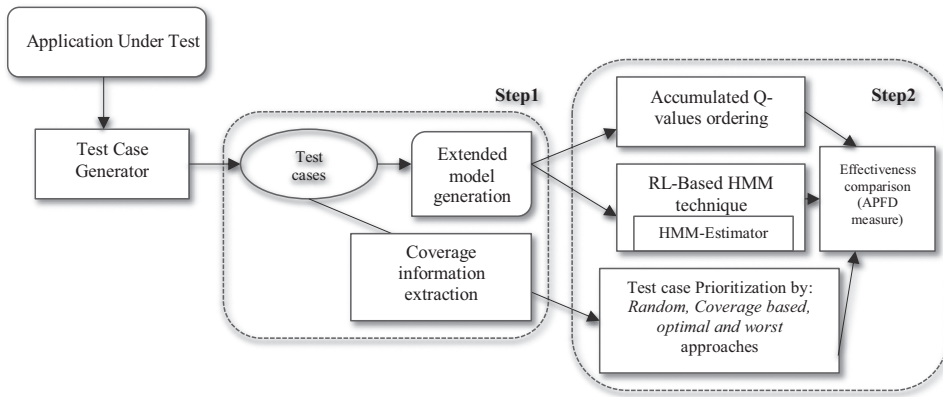


Fig. 7. Experimental setup: combination of a test case generator (ABT) and prioritization techniques.

Table I. Investigated Applications

Application	Version	Statement Cov.(%)	KLOC
UPM ⁷ (a personal password manager)	V 1.6	86	2.515
Buddi ⁸ (a finance tool)	V 3. 4. 0. 8	64	10.580
PDFSAM ⁹ (a merging and splitting PDF documents tool)	V 0.7 Stable release 1	68	3.138
TimeSlotTracker ¹⁰ (tasks and activities management tool)	V 0.4	59	3.499

is generated as a Dot file¹¹. Then, extract the initial HMM parameters and coverage information.

Step2. Train the most appropriate HMM¹² with a maximum likelihood estimate of the parameters. Prioritize test cases based on RL-based HMM and accumulated Q-value ordering techniques, as well as rank them through random, optimal, worst, and additional statement coverage approaches. Finally, analyze the effectiveness of the considered prioritization techniques based upon the APFD measure.

To empirically evaluate AutoBlackTest, widgets from four different domain applications were extracted [Becce et al. 2012] and tested. These applications are listed in Table I and reused in this study, as it allows to use Mariani et al. [2012] as an initial reference point. Mariani et al. [2012] provide results after their test case selection algorithm runs, hence it can be considered as a control case (base value for improvement). In subsequent experiments, we also consider an extended version of PDFSAM with a larger-size test suite and three additional GUI applications from Huang et al. [2010].

7.2.2. Fault Matrix. In a fault matrix, rows represent the test cases and columns represent the faults in the system under test. If test case i detects fault j , then the entry i,j in the matrix would be equal to 1.

So, using a fault matrix, we would be able to indicate the capability of each test case in detecting faults by counting the number of 1's in the corresponding row [Thomas et al.

⁷<http://upm.sourceforge.net/>.

⁸<http://buddi.digitalcave.ca/>.

⁹<http://sourceforge.net/projects/pdfsam/>.

¹⁰<http://sourceforge.net/projects/timeslottracker/>.

¹¹DOCument Template file format.

¹²RHMM package [Tramasco and Bauer 2013] of the R programming environment [Ihaka and Gentleman 1996] has been used.

Table II. Fault Matrix Summary

Application	# Test cases	# Faults	Avg. faults per test	% tests detecting X% faults		
				X = 0	X = 20	X = 50
UPM	15	8	0.5	46.6	0.0	0.0
Buddi	15	2	0.5	86.6	0.0	0.0
PDFSAM	26	3	0.5	88.4	0.0	0.0
TimeSlotTracker	19	1	0.5	94.7	0.0	0.0
Extended PDFSAM	60	6	0.5	90.0	0.0	0.0
WordProcessor	18	12	2.7	72.2	16.6	0
TerpPaint	23	12	3	82.6	8.6	4.3
Calculator	15	12	1.5	40.0	0.0	0.0

2012]. In this research, we have applied several test case prioritization approaches on 8 different GUI applications. Hence, providing the details of all fault matrices in this article is not possible; Table II summarizes the various characteristics of each fault matrix.

7.3. Experimental Results

7.3.1. Study 1: UPM. In this experiment, we investigate the effectiveness of utilizing the considered prioritization techniques on the UPM application. To estimate a suitable RL-based HMM for UPM, the following parameters are needed.

- N*. This is the number of hidden states, which is equivalent to the total number of states in the UPM behavioral model, 52 for this package.
- M*. This is the number of observation symbols, which is equivalent to the number of distinct actions in UPM, that is, 36.
- The state transition probability matrix is calculated using the estimate of Q-values between distinct states (if there is more than one possible action between states, the action with highest Q-value would be selected); in this case, we have a 52×52 transition probability matrix.
- Calculate the frequency of observing a specific action in a specific state; this produces a 52×36 matrix. This matrix is the observation symbol probability matrix.

At the next step, we use the forward algorithm on UPM's hidden Markov model to find the probability of an observed sequence given an estimated HMM. It exploits recursion in the calculations to avoid the necessity for exhaustive calculation of all paths through the execution process. According to Definition 5.1, we prioritized test cases based upon their corresponding F_p .

Therefore, in this case, we calculate 15 F_p for 15 UPM episodes (test cases). Obviously, the accumulated forward probabilities tend to get small (converge to 0) since they are affected by previous probabilities in the forward and recursive process (numerical stability is therefore guaranteed by the scaling introduced in Section 3.2.1).

In addition, it is worthwhile to mention that the summation of the probabilities will not be 1, for the reasons that follow.

- The generated graph is not a complete graph, as there will be distinct vertices that cannot be connected by a pair of unique edges. So some paths cannot be considered.
- Some paths are impossible to walk through because of the application design. Auto-BlackTest only explores the paths (episodes) which are reachable and logical. In this case, 15 episodes are the minimum number of paths that can detect failures.

Therefore, only when we are able to compute the forward probabilities for all possible episodes (paths) of a graph (preferably, a complete graph) can we expect the summation of the forward probabilities to converge to 1. According to the F_p value for each episode,

Table III. APFD of the Applied Prioritization Techniques for UPM

<i>Prioritization Techniques</i>	<i>Means of APFDs</i>	<i>Standard Deviation (SD)</i>
RL-based HMM	0.6865	0.00012
Accumulated Q-value	0.6205	0.00472
Additional Statement Coverage	0.6198	0.0046
Random	0.5018	0.5018
Worst	0.2733	0
Optimal (Best)	0.736	0

the probabilities can be sorted in the following descending order (E_{10} denotes the Episode (test case) number 10):

$$\{F_p(E_{10}) > F_p(E_6) > F_p(E_3) > F_p(E_{14}) > F_p(E_1) > F_p(E_{13}) > F_p(E_9) > F_p(E_2) > F_p(E_4) > F_p(E_7) > F_p(E_8) > F_p(E_{12}) > F_p(E_0) > F_p(E_{11}) > F_p(E_{15})\}.$$

AutoBlackTest reports that Episodes 2, 3, 6, 7, 8, 10, 13, and 14 are able to detect faults in UPM [Mariani et al. 2012]. However, according to its test case selection policy, it only selects Episodes 4, 8, 11, and 13 and eliminates the others. Eliminating episodes which are able to detect faults may cause unexpected costs and problems in regression testing processes.

On the other hand, the proposed techniques (RL-based HMM and accumulated Q-value) are able to prioritize test cases (episodes) along with increasing the original fault detection rate.

To get more reliable results, we execute each of the techniques 1,000 times. Then we calculate the means and standard deviations of the APFDs to evaluate each prioritization technique with respect to the prioritization objective (improving fault detection rate). Then we select an ordering of test cases with the closest APFD to the mean APFD.

As can be seen from Table III, both proposed techniques have larger mean APFDs (and smaller standard deviations), implying their performance is universally “good” (the reason for getting zero SD for both the worst and best approaches is because there is only one test suite which satisfies the best and worst prioritizations).

Figure 8(a) shows the percentage of detected faults versus the execution sequence of test cases for the prioritized test suites, with mean APFD (average performance). This figure depicts that both the RL-based HMM and accumulated Q-value techniques are able to detect a considerable portion of faults by executing a smaller number of test cases. For example, by executing 85% of test cases, both techniques detect 100% of the faults (8 out of 8 faults). This amount is 75% faults for additional statement coverage, and 87% for the random prioritization technique.

In addition, Figure 8(b) shows the distribution of the 1,000 APFD values using a boxplot. This figure indicates that the APFD distributions for the RL-based HMM and accumulated Q-value approaches do not vary in comparison with the random technique.

Table IV¹³ shows the results of applying the statistical tests and the magnitude of Cohen’s d (effect size), which are used to determine the effectiveness of RL-based HMM in comparison with the other approaches. In this table, we can see that the RL-based HMM technique is better than the additional statement coverage and random prioritization techniques. In addition, the RL-based HMM method prioritizes test cases significantly better than the accumulated Q-value approach. Large Cohen’s d s also prove that there is a significant difference, on average, between the fault detection capability of the proposed techniques and other prioritization approaches. This measure is more than 0.8 (large) for all of the considered braces in this case.

¹³DF denotes the *degree of freedom*.

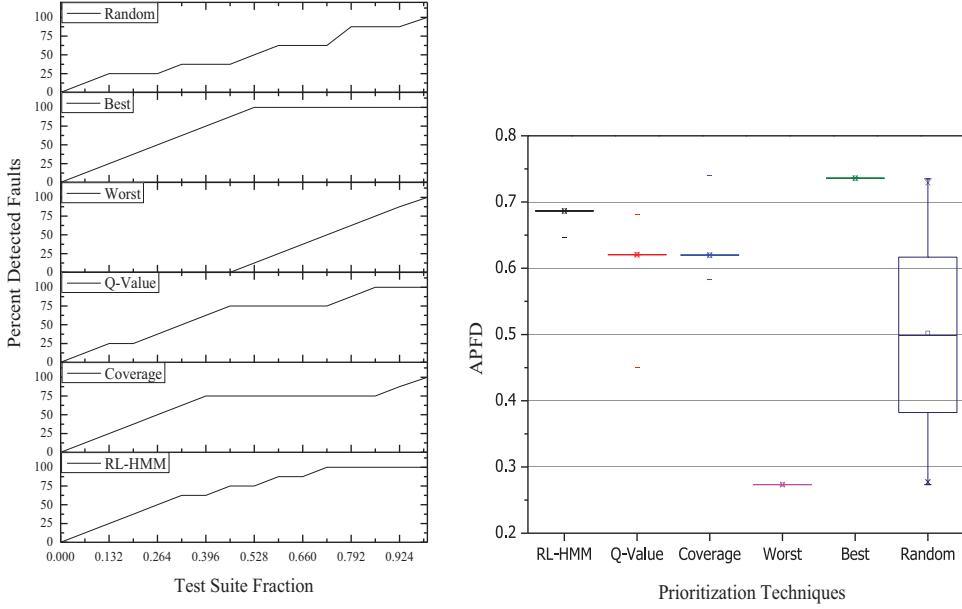


Fig. 8. (a) Percent of faults detected versus test suite fraction; (b) boxplot of APFD for UPM.

Table IV. The Statistical Analysis for RL-Based HMM Technique vs. Other Techniques in UPM

<i>RL-based HMM Technique Vs. Others</i>	<i>t-statistic</i>	<i>DF</i>	<i>p-value (one-tail)</i>	<i>Cohen's d</i>
RL-based HMM vs. Accumulated Q-Value	3.16228	14	0.00644	1.6329
RL-based HMM vs. Statement Coverage	2.23607	14	0.04047	1.1547
RL-based HMM vs. Random	5.36745	14	7.8360E-5	2.7717
RL-based HMM vs. Worst	6.48385	14	1.03159E-5	3.3482
RL-based HMM vs. Optimal (Best)	-2.42272	14	0.02853	-1.2511

We repeat this process for the other three applications to conclude the empirical comparison about the effectiveness of the proposed approaches in improving the fault detection rate.

7.3.2. Study 2. Buddi. To estimate an HMM for the Buddi application, we walk through its behavioral model to compute and extract the following parameters:

- N , the number of hidden states (i.e., 76);
- M , the number of observation symbols (i.e., 45);
- a 76×76 transition probability matrix estimated using Q-values; and
- a 76×45 observation symbol probability matrix.

After achieving the best HMM using the Baum-Welch algorithm, that is, the same process as for the UPM investigation process, was followed, we executed the forward algorithm to compute the forward probability of episodes. Fifteen episodes (F_p) were produced by AutoBlackTest from the Buddi application. Again, we can sort the forward probabilities:

$$\{F_p(E_7) > F_p(E_{14}) > F_p(E_0) > F_p(E_3) > F_p(E_1) > F_p(E_2) > F_p(E_6) > F_p(E_{10}) \\ > F_p(E_5) > F_p(E_9) > F_p(E_8) > F_p(E_{11}) > F_p(E_{12}) > F_p(E_{13}) > F_p(E_4)\}.$$

According to AutoBlackTest [Mariani et al. 2012], Episodes 7 and 14 are considered fault-prone test cases (test cases that reveal faults). The RL-based HMM technique

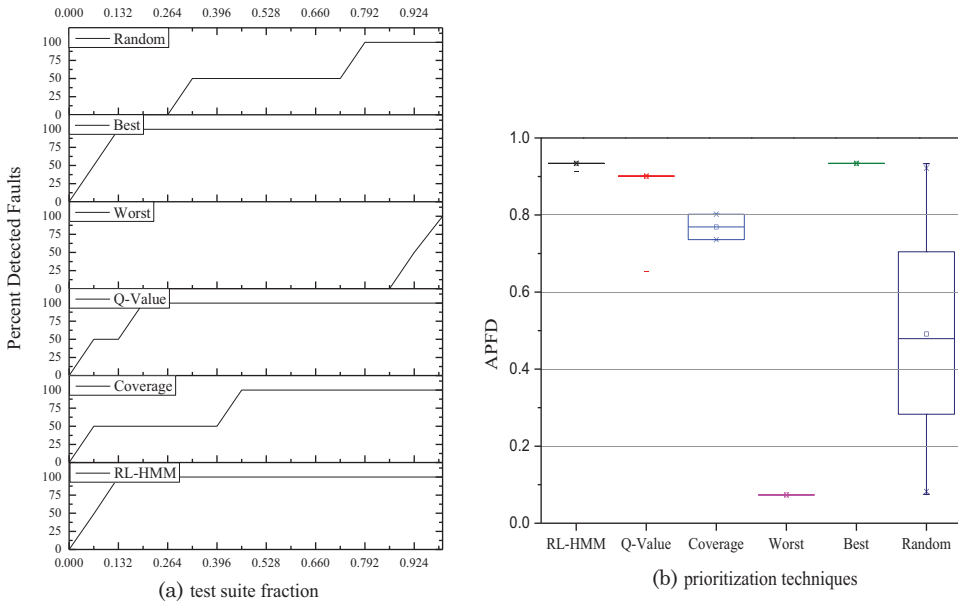


Fig. 9. (a) Percent faults detected versus test suite fraction; (b) boxplot of APFD for Buddi.

Table V. APFD of the Applied Prioritization Techniques for Buddi

Prioritization Techniques	Means of APFDs	Standard Deviation
RL-based HMM	0.9339	7.0698E-4
Accumulated Q-value	0.901	0.0089
Additional Statement Coverage	0.769	0.02688
Random	0.491	0.24868
Worst	0.073	0
Optimal (Best)	0.934	0

correctly prioritizes these two episodes in its final test suite. This test suite is able to find 100% of Buddi's faults in regression testing.

Both proposed techniques are able to detect all the faults by executing only 20% of the test cases, while only one fault can be revealed by executing more than 50% of the test cases using additional statement coverage and random prioritization approaches. This fact is also visible in Figure 9(a), showing the percentage of detected faults using each technique. Also, in Figure 9(b), boxplots summarize the distribution of performance of the prioritization techniques. Again, this distribution shows lower variance for both of the proposed techniques when compared against random and additional statement coverage approaches.

Similarly to UPM, mean and standard deviation of the APFDs for each technique are calculated (Table V). According to this table, and formally verified in Table III, we can conclude that RL-based HMM is not statistically significantly different from the optimal approach (upper bound).

Using Table VI, we provide a statistical statement of RL-based HMM performance when compared with the other techniques. The RL-based HMM technique is significantly different from additional statement coverage and random prioritization, as well as worst prioritization.

Table VI. The Statistical Analysis for RL-Based HMM Technique vs. other techniques in Buddi

<i>RL-based HMM Technique Vs. Others</i>	<i>t-statistic</i>	<i>DF</i>	<i>p-value (one-tail)</i>	<i>Cohen' d</i>
RL-based HMM vs. Accumulated Q-Value	1	14	0.33317	0.5163
RL-based HMM vs. Statement Coverage	2.61116	14	0.01966	1.3483
RL-based HMM vs. Random	4.86926	14	2.04238E-4	2.5144
RL-based HMM vs. Worst	7.45575	14	2.02796E-6	3.8501
RL-based HMM vs. Optimal (Best)	-1	14	0.34463	-0.5163

The Cohen's d measure also indicates that the difference between RL-based HMM and other techniques is large, while medium for RL-based HMM vs. accumulated Q-value, although the difference is not considered statistically significant.

There is a same story for the accumulated Q-value prioritization technique. It prioritizes test cases significantly better than other techniques, except, obviously, for RL-based HMM.

7.3.3. Study 3: PDFSAM. PDFSAM is a Java-based tool for merging and splitting PDF files. Again, the package was used to evaluate the proposed techniques [Mariani et al. 2012]. The initial HMM parameters for PDFAM are:

- N , the number of hidden states, is 79;
- M , the number of observation symbols is 49;
- The transition probability matrix has 79 rows and 79 columns; and
- the observation symbol probability matrix has 79 rows and 43 columns.

In this case, we considered 26 episodes which, according to AutoBlackTest [Mariani et al. 2012], Episodes 0, 2, and 25 are able to detect faults. Ordering the forward probabilities demonstrates that the RL-based HMM prioritization technique prioritizes episodes according to their contribution in fault detection, and no test cases (episodes) are withdrawn.

$$\{F_p(E_{10}) > F_p(E_0) > F_p(E_2) > F_p(E_7) > F_p(E_{11}) > F_p(E_{25}) > F_p(E_{14}) > F_p(E_{15}) \\ > F_p(E_{18}) > F_p(E_6) > F_p(E_3) > F_p(E_{24}) > F_p(E_{20}) > F_p(E_4) > F_p(E_9) > F_p(E_1) \\ > F_p(E_{19}) > F_p(E_5) > F_p(E_{22}) > F_p(E_8) > F_p(E_{17}) > F_p(E_{13}) > F_p(E_{21}) \\ > F_p(E_{12}) > F_p(E_{16}) > F_p(E_{23})\}.$$

This prioritization is able to detect 100% of the faults in PDFSAM's regression testing process by only executing 23% of test cases. The accumulated Q-value approach is only able to detect 67% of faults by executing 78% of prioritized test cases. Figure 10(a) shows the number of detected faults versus test case execution. The percentage of faults detected by the additional statement coverage technique is 33% after executing 42% of the test cases. Also, the APFD distribution (Figure 10(b)) for both proposed techniques is relatively invariant when compared to the random and additional statement coverage prioritizations.

Table VII lists the mean and standard deviations of the APFDs for each test case prioritization technique. Table VIII shows that the RL-based HMM technique is significantly better than the others. The accumulated Q-value technique is significantly better than the additional statement coverage technique. Moreover, again, all the differences between the RL-based HMM and the other techniques are large according to Cohen's d measure.

7.3.4. Study 4: TimeSlotTracker. TimeSlotTracker is the fourth and last application that has been investigated in the evaluation phase. Similar to previous applications, we need to initialize the HMM and finally estimate it with the best parameters to meet the maximum likelihood condition:

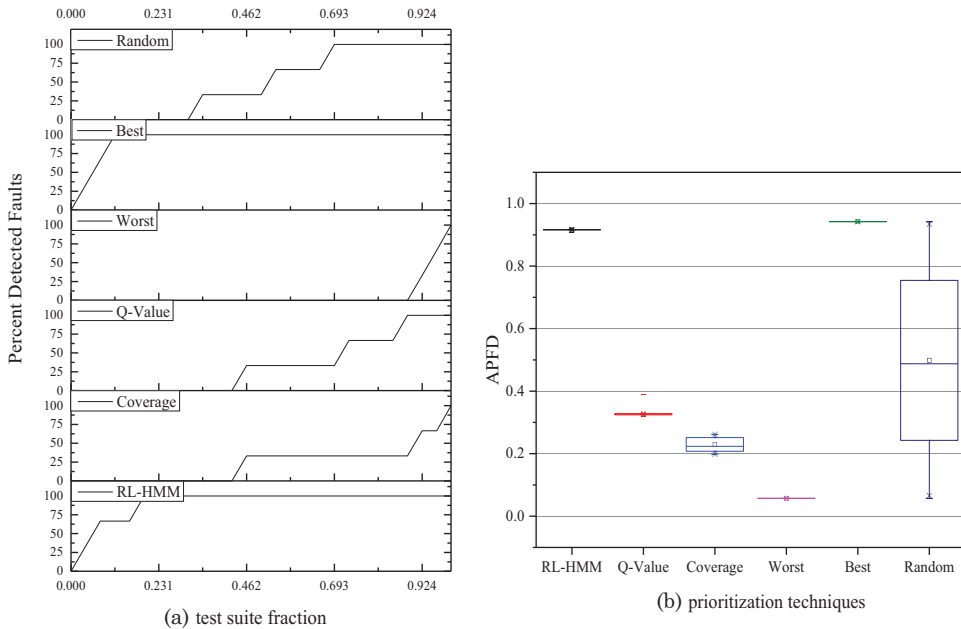


Fig. 10. (a) Percentage faults detected versus the test suite fraction; (b) boxplot of APFD for PDFSAM.

Table VII. APFD of the Applied Prioritization Techniques for PDFSAM

Prioritization Techniques	Means of APFDs	Standard Deviation
RL-based HMM	0.9164	3.1868E-4
Accumulated Q-value	0.3260	0.00206
Additional Statement Coverage	0.2285	0.0217
Random	0.4985	0.25259
Worst	0.0568	0
Optimal (Best)	0.9422	0

Table VIII. The Statistical Analysis for RL-Based HMM Technique vs. Other Techniques in PDFSAM

RL-based HMM Technique Vs. Others	<i>t</i> -statistic	DF	<i>p</i> -value (one-tail)	Cohen' <i>d</i>
RL-based HMM vs. Accumulated Q-Value	8.2955	25	8.9005E-9	3.2537
RL-based HMM vs. Statement Coverage	12.49415	25	1.70486E-12	4.9006
RL-based HMM vs. Random	5.6673	25	5.83228E-6	2.2228
RL-based HMM vs. Worst	13.78585	25	1.82592E-13	5.4072
RL-based HMM vs. Optimal (Best)	-1.44222	25	0.16118	-0.5656

- N , the number of hidden states, is 246;
- M , the number of observation symbols, is 46;
- a 246×246 matrix represents the transition probability matrix; and
- the observation symbol probability matrix has 246 rows and 46 columns.

In this case, AutoBlackTest generates 19 episodes throughout the testing process and Episode 18 is considered as the only fault-revealing test case. Estimating an appropriate RL-based HMM, we compute the forward probabilities as before.

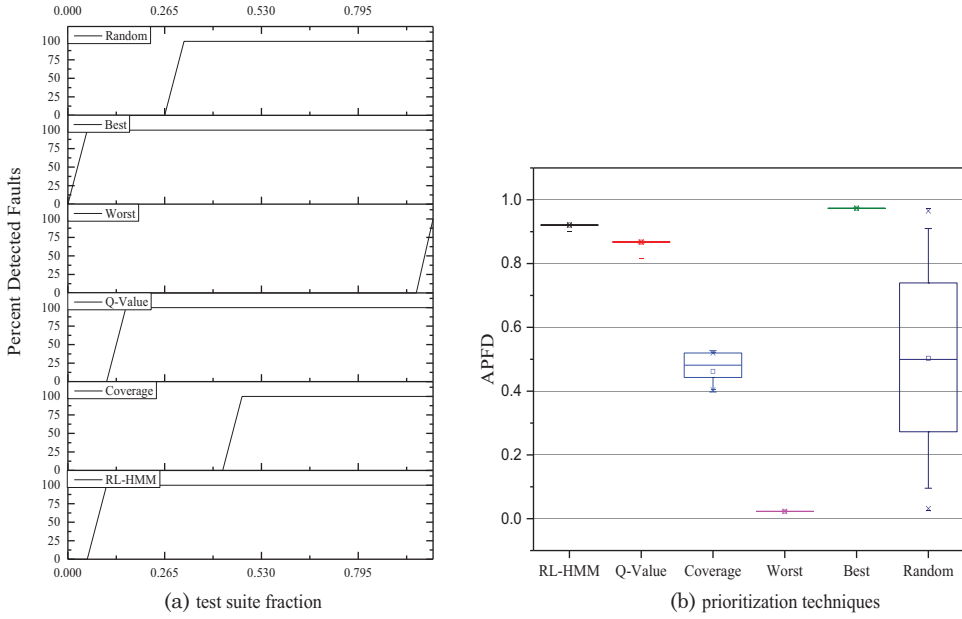


Fig. 11. (a) Percentage faults detected versus test suite fraction; (b) boxplot of APFD for TimeSlotTracker.

After sorting the probabilities, the RL-based HMM prioritization technique reports the following test suite.

$$\{F_p(E_{18}) > F_p(E_5) > F_p(E_7) > F_p(E_0) > F_p(E_{12}) > F_p(E_1) > F_p(E_9) > F_p(E_6) \\ > F_p(E_{11}) > F_p(E_4) > F_p(E_{10}) > F_p(E_{15}) > F_p(E_{14}) > F_p(E_{13}) > F_p(E_3) \\ > F_p(E_8) > F_p(E_2) > F_p(E_{16})\}.$$

This shows that the fault detection rate using prioritized test cases would be 100% for TimeSlotTracker; it means that, by executing only one test case, the fault would be detected. On the other hand, this fault would be found after the execution of 21% and 37% of the test cases, respectively, using accumulated Q-value and additional statement coverage approaches. Figure 11(a) also depicts the percentage of detected faults versus test case executions for each applied prioritization technique. In addition, the additional statement coverage and random approaches still have more varying distributions than the other techniques, while the RL-based HMM distribution is “close” to the upper bound (Figure 11(b)). The mean and standard deviation APFD of the prioritization processes are shown in Table IX. Table X indicates that RL-based HMM is significantly better than random and additional statement coverage, but not significantly different from the accumulated Q-value prioritization method. Cohen’s d measure also confirms this result. The accumulated Q-value approach again provides better performance than the additional statement coverage technique.

7.3.5. Study 5: Extended PDFSAM. To demonstrate that the proposed techniques are also applicable to prioritizing test cases with somewhat larger-size test suites, we applied the approaches to order 60 test cases.

For this study, we run the testing process (AutoBlackTest) of PDFSAM again, but now for two hours. The default testing time had been set to 1 hour for the other case studies in order to allow an unbiased comparison across application programs. In the other case studies except for PDFSAM, the test case generation procedure completed

Table IX. APFD of the Applied Prioritization Techniques for TimeSlotTracker

<i>Prioritization Techniques</i>	<i>Means of APFDs</i>	<i>Standard Deviation (SD)</i>
RL-based HMM	0.9209	7.2276E-4
Accumulated Q-value	0.8675	0.0017
Additional Statement Coverage	0.462	0.04305
Random	0.5028	0.27145
Worst	0.023	0
Optimal (Best)	0.9735	0

Table X. The Statistical Analysis for RL-based HMM Technique vs. Other Techniques in TimeSlotTracker

<i>RL-based HMM Technique Vs. Others</i>	<i>t-statistic</i>	<i>DF</i>	<i>p-value (one-tail)</i>	<i>Cohen's d</i>
RL-based HMM vs. Accumulated Q-Value	1	18	0.32988	0.4588
RL-based HMM vs. Statement Coverage	2.51661	18	0.02099	1.1546
RL-based HMM vs. Random	2.17946	18	0.04209	1.000004
RL-based HMM vs. Worst	10.37642	18	2.89352E-9	4.7610
RL-based HMM vs. Optimal (Best)	-1	18	0.32988	-0.4588

before the testing time finished. In this experiment, we generated 60 test cases, of which Test Cases 0, 2, 25, 29, 40, and 52 were able to detect faults. In order to initialize the HMM and finally estimate it with the best parameters to meet the maximum likelihood condition, we consider the following parameters:

- N , the number of hidden states, is 207;
- M , the number of observation symbols, is 79;
- a 207×207 matrix represents the transition probability matrix; and
- the observation symbol probability matrix also has 207 rows and 79 columns.

After estimating the model and calculating the accumulated Q-values, we sort the test cases based upon each considered technique. The results show that the RL-based HMM technique again outperforms the other techniques when considering improving the fault detection rate. For instance, a prioritized test suite using RL-based HMM is able to detect all 6 faults by executing only 16 test cases (only 26% of the test suite), while the statement coverage technique needs to run 48 test cases (80% of the test suite) to find all of the faults. Figure 12(a) shows the percentage of detected faults versus the fraction of test suites executed with the mean APFD. Figure 12(b) also shows that both proposed techniques are relatively invariant in terms of APFD.

In addition, Table XI illustrates the results of computing the mean and standard deviation of the APFD measure for 1,000 trials per application. Table XII reports the result of applying a paired t-test and calculating Cohen's d measure on the percentage of detected faults by executing a prioritized test suite with mean amount of APFD. According to this investigation, the RL-based HMM technique prioritizes test cases significantly better than the other techniques, except for optimal. Also, the accumulated Q-value technique has better APFD measure than coverage-based prioritization and sorts test cases significantly better.

7.3.6. Study 6: WordProcessor, TerpPaint and Calculator. As in our last experiment, we have applied our proposed techniques on three GUI applications: WordProcessor, TerpPaint, and Calculator¹⁴. TerpPaint is a paint program written using Java and AWT/Swing. It is similar to Microsoft Paint and developed at the University of Maryland. WordProcessor is a small application developed by Robinson and Vorobiev [1999] as part of their effort in a “fast-paced guide” for teaching Swing. Finally, Calculator is an open-source

¹⁴<http://sourceforge.net/projects/terppaint>.

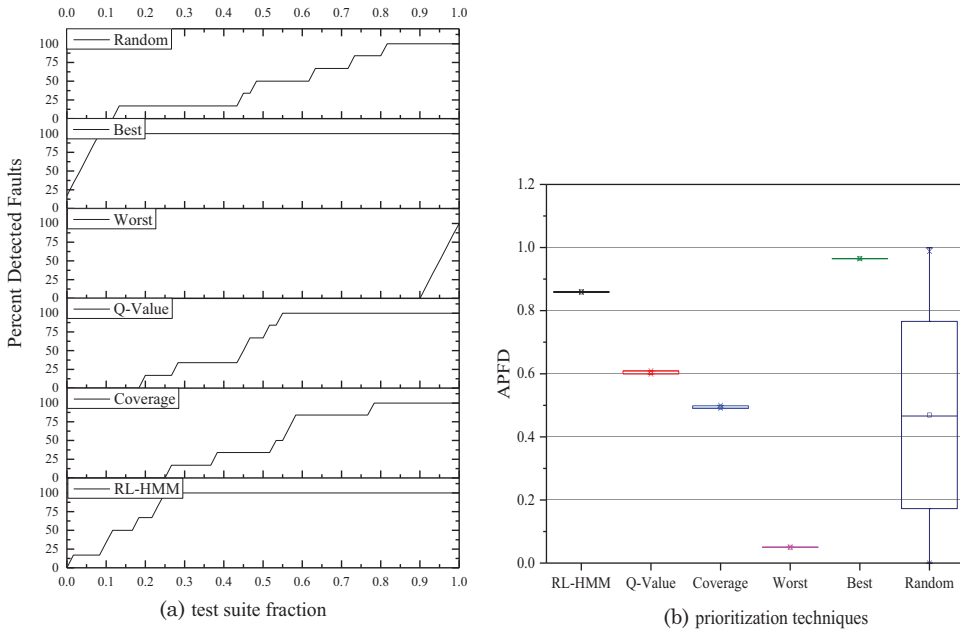


Fig. 12. (a) Percentage faults detected versus test suite fraction; (b) boxplot of APFD for extended PDFSAM.

Table XI. APFD of the Applied Prioritization Techniques for Extended PDFSAM

Prioritization Techniques	Means of APFDs	Standard Deviation (SD)
RL-based HMM	0.895	0
Accumulated Q-value	0.604	0.005
Additional Statement Coverage	0.494	0.0042
Random	0.469	0.2967
Worst	0.0503	0
Optimal (Best)	0.965	0

Table XII. The Statistical Analysis for RL-Based HMM Technique vs. Other Techniques in Extended PDFSAM

RL-based HMM Technique Vs. Others	t-statistic	DF	p-value (one-tail)	Cohen' d
RL-based HMM vs. Accumulated Q-Value	6.89838	59	3.72746E-9	1.7961
RL-based HMM vs. Statement Coverage	8.91787	59	1.35721E-12	2.3220
RL-based HMM vs. Random	10.16044	59	1.16392E-14	2.6455
RL-based HMM vs. Worst	19.28205	59	2.13086E-27	5.0206
RL-based HMM vs. Optimal (Best)	-3.94484	59	2.1174E-4	-1.0271

program which implements a four-function calculator; Calculator is developed using Java Swing¹⁵.

The purpose of choosing these applications is to compare the results of our test case prioritization approaches against those presented in Huang et al. [2010]. Unfortunately, the framework used in their experiment is not available. Further, the paper's explanations are insufficient to re-implement their experimental environment.

Hence, we can only use the paper's results and run our methods on identical applications. This implies that the results are not directly comparable, as the test data will be

¹⁵<http://beginner-java-tutorial.com/java-swing-calculator.html>.

Table XIII. APFD of the Applied Prioritization Techniques for TerpPaint

<i>Prioritization Techniques</i>	<i>Means of APFDs</i>	<i>RR</i>
RL-based HMM	0.924	0.0149
Accumulated Q-value	0.891	0.0501
Additional Statement Coverage	0.755	0.1950
Untreated	0.690	0.2643
Best	0.938	0
High-to-low equal weight (L1)	0.934	0.603
High-to-low fault-prone weight (L1)	0.844	0.1590
High-to-low adjusted weight (L1)	0.966	0.0281
Best (L1)	0.994	0

different in the two studies. Since AutoBlackTest generates meaningful test cases with optimum lengths, combining test cases to generate different-length test suites can lead to generating false positive fault-revealing test cases. Therefore, we only consider the prioritization results for L1 test cases from Huang et al.'s [2010] research.

In order to increase the similarities between the two experimental frameworks, we also seeded 12 faults from the same types into a random location in each application and ran (100 times) RL-based HMM, accumulated Q-value, and coverage techniques on each set of test cases generated using AutoBlackTest. In addition, the untreated (unprioritized) and best orderings of test cases have been considered to estimate the test effectiveness boundaries, and hence will be used as the point of comparison.

It is worth noting that the reason for considering the untreated ordering in this research is that Huang et al.'s [2010] research only covers two techniques (untreated and coverage) as nonweight-based prioritization methods and applies t-tests and statistical analysis on them. Also, they didn't compare weight-based techniques with the best and worst orderings. Therefore, unlike the previous studies, we have compared our proposed technique with the untreated method in order to follow the same approach as theirs.

Because the exact test cases used in Huang et al.'s [2010] research were not available for doing an accurate comparison, it cannot be claimed that the test cases generated and used in this research have the exact same coverage or fault detection capability as Huang et al.'s [2010] test cases. Therefore, we have decided to investigate the effectiveness of each considered technique by comparing its fault detection capability with untreated, coverage-based, and best orderings. In order to clarify the effectiveness of each test case prioritization technique, we also compared the mean of APFD of each method with the best prioritization (the optimal ordering of test cases). As the sets of test cases are different in the two works, the best (or optimal) results are different. The following formula shows the method we have used to calculate the relative ratio of each method versus Huang et al.'s [2010] best (optimal) approach.

$$Relative\ Ratio\ (RR) = \frac{Best_{APFD} - Treatment_{APFD}}{Best_{APFD}} \quad (16)$$

Obviously, prioritization techniques with a low RR value approach optimality. Tables XIII–XVII show the mean APFDs and relative ratios for each applied technique. In addition, the results of the statistical analysis for each considered application are provided.

Table XIII indicates that the RL-based HMM technique has behavior closest to the best ordering, due to the low RR (=0.0149). In addition, according to Table XIV, it also prioritizes test cases significantly better than untreated and coverage-based prioritizations in TerpPaint. The same prioritization efficiency can be interpreted based upon Tables XV–XVIII for both the WordProcessor and Calculator applications. For example

Table XIV. The Statistical Analysis for RL-Based HMM Technique vs. Other Techniques in TerpPaint

<i>RL-based HMM Technique Vs. Others</i>	<i>t-statistic</i>	<i>p-value (one-tail)</i>
RL-based HMM vs. Statement Coverage	3.81329	8.93385E-4
RL-based HMM vs. Untreated	4.12466	4.12614E-4
RL-based HMM vs. Best	1.2916	0.20932
Untreated vs. fault-prone weight (L1)	2.6730	0.0043
Untreated vs. adjusted weight (L1)	-3.1059	0.0012
Coverage vs. fault-prone weight (L1)	3.6082	0.0002
Coverage vs. fault-prone weight (L1)	-1.9221	0.0286

Table XV. APFD of the Applied Prioritization Techniques for WordProcessor

<i>Prioritization Techniques</i>	<i>Means of APFDs</i>	<i>RR</i>
RL-based HMM	0.899	0.0088
Accumulated Q-value	0.887	0.0220
Additional Statement Coverage	0.716	0.2105
Untreated	0.707	0.2205
Best	0.907	0
High-to-low equal weight (L1)	0.946	0.0463
High-to-low fault-prone weight (L1)	0.969	0.0231
High-to-low adjusted weight (L1)	0.972	0.0201
Best (L1)	0.992	0

Table XVI. The Statistical Analysis for RL-Based HMM Technique vs. Other Techniques in WordProcessor

<i>RL-based HMM Technique Vs. Others</i>	<i>t-statistic</i>	<i>p-value (one-tail)</i>
RL-based HMM vs. Statement Coverage	4.265	4.65314E-4
RL-based HMM vs. Untreated	3.67374	0.00174
RL-based HMM vs. Best	-1.0063	0.327
Untreated vs. fault-prone weight (L1)	-3.7138	0.0001
Untreated vs. adjusted weight (L1)	-3.6246	0.0002
Coverage vs. fault-prone weight (L1)	-0.6824	0.2485
Coverage vs. fault-prone weight (L1)	-0.7929	0.2151

(Table XV), the RL-based HMM approach prioritizes test cases with the lowest difference compared to the best technique ($RR = 0.0088$), while for the high-to-low adjusted weight (L1), the most accurate proposed method by Huang et al. [2010], the RR value is equal to 0.0201.

8. DISCUSSION

We have investigated four different applications by inferring four different RL-based hidden Markov models. The results show that the RL-based HMM prioritization technique is successful in all four cases, plus a second trial on PDFSAM, plus three extra GUI applications proposed by Huang et al. [2010], specifically in prioritizing test cases that contribute to discovering faults.

According to the empirical results represented in Section 7, since the RL-based HMM technique worked significantly better than other applied approaches according to multiple statistical tests, effect-size measures, and more informal investigations, the first research question (RQ1) defined in the beginning of this section has been addressed. In addition, according to Table II, we can conclude that, even though in some cases (like UPM) other approaches (such as additional statement converge and random) have an

Table XVII. APFD of the Applied Prioritization Techniques for Calculator

<i>Prioritization Techniques</i>	<i>Means of APFDs</i>	<i>RR</i>
RL-based HMM	0.708	0.0432
Accumulated Q-value	0.622	0.1594
Additional Statement Coverage	0.577	0.2202
Untreated	0.408	0.4480
Best	0.740	0
High-to-low equal weight (L1)	0.838	0.1440
High-to-low fault-prone weight (L1)	0.898	0.0827
High-to-low adjusted weight (L1)	0.931	0.0490
Best (L1)	0.979	0

Table XVIII. The Statistical Analysis for RL-Based HMM Technique vs. Other Techniques in Calculator

<i>RL-based HMM Technique Vs. Others</i>	<i>t-statistic</i>	<i>p-value (one-tail)</i>
RL-based HMM vs. Statement Coverage	3.81329	8.93385E-4
RL-based HMM vs. Untreated	4.12466	4.12614E-4
RL-based HMM vs. Best	-2.99924	0.00899
Untreated vs. fault-prone weight (L1)	-3.3235	0.0013
Untreated vs. adjusted weight (L1)	-3.8280	0.0003
Coverage vs. fault-prone weight (L1)	-2.4221	0.0113
Coverage vs. fault-prone weight (L1)	-3.2269	0.0016

acceptable performance, RL-based HMM is more successful in detecting difficult-to-find faults. For example, in TimeSlotTracker where the chance of finding the fault is 0.05, RL-based HMM is able to correctly prioritize the fault-revealing test case.

To address the second research question (RQ2), we replicated all the experiments and analysis for another proposed technique (accumulated Q-value). This technique achieves results demonstrating its better mean APFD than the other techniques (except RL-based HMM and optimal), and that it's significantly better than random and additional coverage when applied to the PDFSAM and Buddi applications.

The third question (RQ3) compares the effectiveness of both proposed techniques. As mentioned earlier, statistical analysis illustrates that the RL-based HMM is significantly more effective than the accumulated Q-value technique in 3 of the 4 applications, although in the fourth (TimeSlotTracker) application they are not significantly different. The results from the TimeSlotTracker experiment demonstrate a (worst case) situation where the effect of executing actions (causing further computations) in a specific (GUI) state is similar to the effect of triggering different actions in each state. That is, they contribute the "same amount" of information for making prioritization decisions. The TimeSlotTracker application is a good example of an application, having only actions which trigger small amounts of computation. Under these circumstances, the performance difference between the RL-based HMM technique and the accumulated Q-value approach will be minimal. However, it is believed that the RL-based HMM technique will always outperform the accumulated Q-value approach.

In addition, in order to investigate the effectiveness of the proposed techniques in cases with larger test suite sizes, we ran AutoBlackTest for an extended period and reapplied all the techniques on a test suite now with 60 test cases. The results show that both of our approaches are still applicable to larger-sized test suites, and that they also prioritize them better than the other state-of-the-art methods.

To answer the fourth question (RQ4), we also have considered three GUI-based applications from Huang et al. [2010] in order to show the effectiveness of our proposed technique in comparison with other GUI test case prioritization approaches. The results

demonstrate that, in all the applications, the RL-based HMM technique is significantly better than untreated and coverage-based prioritizations, while only the fault-prone weight method outperforms coverage-based and untreated orderings in terms of fault detection rate for the TrepPaint application. In addition, the RL-based HMM is not significantly different from the best ordering in all considered applications according to the relative ratio measure.

It is believed these results illustrate that the combination of reinforcement learning and hidden Markov models can be highly successful in prioritizing test cases because this enhanced model tends to consider the test cases from two different and novel perspectives: prioritizing test cases that: (a) trigger different actions in each state, and (b) execute actions causing more computation in GUI states.

8.1. Runtime Analysis

To investigate the computational complexity of the RL-based HMM prioritization approach, we ran a single instance of this technique on a modest hardware and software platform consisting of a 3.4 GHz CPU, 8GB RAM on an HP Compaq machine (6200 Pro SFF PC) using the RHMM package which has been developed using the R language (R is a statistical scripting language¹⁶) for estimating HMM parameters.

In order to investigate the systems with the largest number of test cases generated by AutoBlackTest and GUITAR, we have considered both the extended PDFSAM and TerpPaint applications in terms of prioritization runtime. The RL-based HMM technique ran in 12.3s in extended PDFSAM (8.1s to extract the Q-values and initial values of the HMM parameters, 3.26s for estimating an HMM using the Baum-Welch algorithm, and less than a second to calculate the forward probabilities). Also, based upon Baum-Welch and forward algorithms' time complexities, the computation order is polynomial $O(N^2T)$, where N represents the number of hidden states and T indicates the number of observations in a sequence; hence, this will not exponentially grow by increasing the number of test cases. The corresponding time for the TerpPaint application is 17.1s. It is worth noting that the parameter extraction techniques used in this study can be optimized in terms of implementation (e.g., anecdotal comparisons often state that algorithms written in R run 1,000 times slower than equivalent algorithms in C¹⁷) to decrease the overall runtime in future studies.

9. RELATED WORK

Applying Markov chain models in software testing processes dates back to 1994, when Whittaker and Thomason [1994] described a method for statistical software testing. They used Markov chains as a finite-state, discrete-parameter, and time-homogeneous modeling approach to develop and analyze a model-based testing technique for automatic test case generation. They concluded that Markov chain usage models can be utilized in a diverse set of application domains and are useful during a statistical testing process. Correspondingly, Markov chain usage models have been utilized several times in software testing and reliability research [Semmel and Linton 1998; White and Sjorgen 1988; Zhen and Chenglian 2004]. Also Bayesian network models have been applied in software failure detection problems to design software reliability models [Bai et al. 2005], or in probabilistic test case prioritization to incorporate source-code changes and test coverage data into a unified model [Mirarab and Tahvildari 2007, 2008]. Moreover, Web application testing is becoming another interesting area for applying Markov chain models. Hao and Mendes [2006] considered the effectiveness of *unified Markov models* (UMM) as a suitable testing mechanism through two empirical

¹⁶<http://www.r-project.org>.

¹⁷<http://lists.nongnu.org/archive/html/igraph-help/2011-02/msg00045.html>.

studies. They confirmed that UMMs are appropriate methods to test Web applications accurately. Cai et al. [2005] present a *controlled Markov chain* (CMC) approach to software testing. They considered software testing as a control problem where the software under test serves as a controlled object. Avritzer and Weyuker [1995] also presented an MBT method to generate test cases to perform load testing for any software system that can be modeled by a Markov chain. Their algorithm was successful in preserving faults that would have been likely missed in traditional load testing methods. The purpose of the authors in designing this technique was not test case prioritization. However, their thoughts are statistically close to what is presented in this research.

One of the most critical parameters in using Markov chain models in the MBT area is in estimating transition probabilities. Much research provided diverse techniques to estimate Markov chain parameters, however, Feliachi and Guen [2010] proposed a novel technique to target coverage criteria rather than using a classical uniform probability generation approach. It would be an applicable technique in creating a Markov chain with appropriate initial parameters leading to a globally optimized model.

As mentioned earlier, the application of HMM models to pattern recognition and bioinformatics problems has been well established. On the other hand, limited numbers of papers have been published on *hidden Markov models* (HMMs) applied to software testing; however, it has been used for modeling computer security problems and intrusion detection in Web applications. HMMpayl is a combination of HMM usage and multiple classifier systems that has been reported as an effective tool against the most frequent attacks on Web applications [Ariu et al. 2011].

In addition, combining HMM and reinforcement learning concepts to re-estimate and improve the model has been considered in several artificial intelligence problems such as robot motion prediction [Hamamoto et al. 2004], speech recognition [Walker 2000], and natural language generation [Cuay and Dethlefs 2011]. Also, Jaakkola et al. [1995] suggest a method to handle RL algorithms in partially observable Markov decision problems. McCallum [1995] provides a comprehensive study on RL with hidden states. He also represents four new RL algorithms for environments with hidden crucial states.

One of the reasons, which initially motivated the ideas beyond the design of an RL-based HMM test case prioritization technique, was the existence of prioritization approaches only applicable for non-GUI testing. Rothermel et al. [2001] describe several techniques for using information gathered from the test case generation phase to prioritize test cases in regression testing. They measured the fault detection rate of each method and found their proposed techniques are able to improve this rate. Furthermore, Yu and Lau [2012] present a novel test suite prioritization technique which concentrates on test cases' fault detecting ability. They used a fault model which generates test cases to guide specification-based testing. Results confirm the effectiveness of this approach in logical expression testing, but there is no evidence its applicability to the testing of state-based models, which are very useful in testing interactive applications. In addition, Do et al. [2010] consider the application of test case prioritization in regression testing procedures from a time-constraint perspective. The authors believe that the time constraints which can be imposed on regression testing can strongly affect the behavior of prioritization techniques. They also provide helpful suggestions about determining the appropriate situation for prioritizing test cases based upon the system's constraints.

Finally, Huang et al. [2010] have proposed a method for adding weights to event-flow graphs (a very primitive version of a regular digraph). They use the GUITAR tool to generate test cases and a behavioral model to which they add weights to the transitions of this graph. This results in a weighted graph similar to the graph produced by AutoBlackTest. While AutoBlackTest dynamically analyses the SUT to produce system-specific weights, Huang et al. [2010] simply use static weights derived

from heuristics which have very limited theoretical underpinnings. This implies that the digraph produced by AutoBlackTest is clearly superior to that by Huang et al. [2010]. Hence, we use the output of AutoBlackTest rather than of Huang et al. [2010] as the baseline for analyzing test case prioritization techniques in interaction-driven applications.

10. THREATS TO VALIDITY

In this section, some potential threats to validity of our research are discussed. We consider two types of threats.

- Threats to internal validity are cause-effect relationships between independent and dependent variables.
- Threats to external validity are limitations about generalizations from our study, or the threats to transforming the research from the specific case study to general subjects.

With respect to internal validity, the most significant threats are: (a) the four different models (one per application) generating diverse levels of forward probabilities; and (b) the size of the test suites derived by AutoBlackTest from these applications. This first threat can be classified as an instrumentation effect. This can add bias into the results of the APFD measures. To reduce the likelihood of this effect, we have normalized the Q-values and APFD measure to within the range $[0, 1]$, thus minimizing any impact on the values of the transition matrices and the APFD measure. The second threat represents the most significant issue. Given the choice of the domain of application—extending the automatic generation of (black box) test cases for GUI-based applications—no obvious approach exists to minimize this threat. AutoBlackTest produces superior results when compared to other systems (such as GUITAR) in this domain of application. However, the volume of test cases it generates can be viewed as “rather small”, leading to prioritization problems of “limited extent”. In fact, AutoBlackTest explicitly discards tests it considers to be of no value, which compounds this “size of test suite” issue further. Hence, further research is required in producing more complete regular digraphs and extended digraphs models to further advance the topic.

On the other hand, the threats to external validity for our research are centered on the selected applications as representative of any possible application. To address this issue, we evaluate our method on four different GUI-based applications. As mentioned before, any application with a weighted or nonweighted behavioral model can be utilized, but, because results show that AutoBlackTest is a successful model-based GUI testing tool, we have concentrated on working in this area. Also, by evaluating the proposed method on known and pre-evaluated applications, we can compare our results and illustrate the effectiveness of the RL-based HMM technique.

11. CONCLUSIONS AND FUTURE WORK

Many studies have investigated automatic test case generation, test case prioritization, and GUI testing, but few specifically focus on the fault-based test case prioritization issue in GUIs.

In this research, we propose a novel fault-based technique, the extended digraph. It is argued that the extended digraph provides a richer explanation of program behavior than a regular digraph. This digraph is generated by using an RL-based HMM approach to prioritize test cases. We present an approach to initialize an appropriate HMM based on a Q-learning algorithm that leads to a final HMM with the maximum likelihood estimate of parameters after applying an EM algorithm. Then we use the estimated model to compute the likelihood (forward probability) of the generated test

cases. Finally, we presented a new definition for test case prioritization based upon these forward probabilities.

In addition, we propose another technique which uses the summation of each test case's Q-value in order to sort them in a descending order. Thus, test cases with higher amount of accumulated Q-value get higher priority in the sorted list.

To evaluate the proposed methods, we designed an experimental setup using AutoBlackTest as a test case generator tool. Random and additional statement coverage, and worst and optimal prioritization are also included in order to provide a comprehensive comparison. We applied all of the prioritization techniques on four GUI applications to evaluate the effectiveness of our proposed approach and to address the first three research questions. In addition, as an extra experiment, we have compared the RL-based HMM technique with the weight-based prioritization approach presented by Huang et al. [2010].

According to the APFD measure, relative ratio, boxplot, statistical tests, and effect-size estimates, RL-based HMM outperforms the other approaches in terms of fault detection effectiveness. It illustrates that considering GUI states and actions plays an important role in improving the fault detection rate. Because the RL-based HMM approach combines RL and HMM concepts, it is able to sort a test suite by prioritizing test cases with special focus on two important aspects:

- the amount of computations (changes) a test case may cause in GUI states (using Q-learning); and
- the probability of triggering each action in each specific state, plus the amount of computations (changes) a test case may cause in GUI states (using HMM).

Although the results of this research improve the effectiveness of test case prioritization, the RL-based HMM approach still has room for improvement. First, we intend to perform additional studies on more applications such as Web-based. Second, in this study we only consider GUI applications. We want to evaluate this method further; we are working on presenting a generic approach to generate an RL-based weighted model for every type of application. Thomas et al. [2012] represented a static approach to prioritizing Junit test cases by defining the distance between pairs of test cases based upon topic modeling. Such techniques can be utilized to compute the reward function and Q-values in non-GUI-based applications. Third, we need the best sequence of GUI states contributing to the most appropriate prioritized test suite. In this situation, we can address the second HMM problem using a Viterbi algorithm to find the most suitable ordering which maximizes the HMM's likelihood of estimated parameters.

ACKNOWLEDGMENTS

The authors would like to acknowledge the AutoBlackTest team (Laboratory of Test and Analysis, University of Milano-Bicocca, Italy, Leonardo Mariani, Mauro Pezzè, Oliviero Riganelli, Mauro Santoro) for providing a copy of AutoBlackTest and special thanks to Oliviero Riganelli for his support during our faltering first steps with AutoBlackTest.

REFERENCES

- K. Aas and L. Eikvil. 1996. Text page recognition using grey-level features and hidden Markov models. *Patt. Recogn.* 29, 6, 977–985.
- A. Arcuri and L. Briand. 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE'11)*. IEEE, 1–10.
- I. Arel, C. Liu, T. Urbanik, and A. G. Kohls. 2010. Reinforcement learning-based multi-agent system for network traffic signal control. *IET Intell. Transport Syst.* 4, 2, 128–135.

- D. Ariu, R. Tronci, and G. Giacinto. 2011. HMMPayl: An intrusion detection system based on hidden Markov models. *Comput. Secur.* 30, 4, 221–241.
- S. Arlt, S. Pahl, C. Berolini, and M. Schaf. 2012. Trends in model-based GUI testing. *Adv. Comput.* 86, 183–222.
- A. Avritzer and E. R. Weyuker. 1995. The automatic generation of load test suites and the assessment of the resulting software. *IEEE Trans. Softw. Engin.* 21, 9, 705–716.
- C. G. Bai, Q. P. Hu, M. Xie, and S. H. Ng. 2005. Software failure prediction based on a Markov Bayesian network model. *J. Syst. Softw.* 74, 3, 275–282.
- L. Baum, T. Petrie, G. Soules, and N. Weiss. 1970. A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *The Ann. Math. Statist.* 41, 1, 164–171.
- G. Becce, L. Mariani, O. Riganelli, and M. Santoro. 2012. Extracting widget descriptions from GUIs. In *Proceedings of the 15th International Conference on Fundamental Approaches to Software Engineering (FASE'12)*. 347–361.
- P. Blunsom. 2004. Hidden Markov models. <http://digital.cs.usu.edu/~cyan/CS7960/hmm-tutorial.pdf>.
- K.-Y. Cai, Y.-C. Li, and W.-Y. Ning. 2005. Optimal software testing in the setting of controlled Markov chains. *Euro. J. Oper. Res.* 162, 2, 552–579.
- H. S. Chang. 2006. Reinforcement learning with supervision by combining multiple learnings and expert advices. In *Proceedings of the American Control Conference (ACC'06)*. 159–161.
- W. Ching. 2006. *Markov Chains: Model, Algorithms and Applications*. Springer Science.
- J. Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associate Publishers.
- H. Cua and N. Dethlefs. 2011. Hierarchical reinforcement learning and hidden Markov models for task-oriented natural language generation. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Short Papers (ACL'11)*. 654–659.
- R. Davoodi and B. J. Andrews. 1998. Computer simulation of FES standing up in paraplegia: A self-adaptive fuzzy controller with reinforcement learning. *IEEE Trans. Rehabil. Engin.* 6, 2, 151–161.
- P. Dayan and C. J. Watkin. 1992. Technical note Q-learning. *Mach. Learn.* 8, 279–292.
- R. Dev, A. Jaaskelainen, and M. Katara. 2012. Model-based GUI testing: Case smartphone camera and messaging development. *Adv. Comput.* 85, 65–122.
- H. Do, S. Mirarab, L. Tahvildari, and G. Rothermel. 2010. The effect of time constraints in test case prioritization: A series of controlled experiments. *IEEE Trans. Softw. Engin.* 36, 5, 592–617.
- I. K. El-Far and J. A. Whittaker. 2002. Model-based software testing. In *Encyclopedia of Software Engineering*. Wiley, 1–22.
- S. Elbaum, G. Rothermel, A. G. Malishevsky, and S. Member. 2002. Test case prioritization: A family of empirical studies test case prioritization. *IEEE Trans. Softw. Engin.* 27, 10, 929–948.
- D. P. Ellis. 2010. *The Essential Guide to Effect Sizes; Statistical Power, Meta-Analysis and the Interpretation of Research Results*. Cambridge University Press.
- A. T. Endo and A. Simao. 2011. Model-based testing of service-oriented applications via state models. In *Proceedings of the IEEE International Conference on Services Computing (SCC'11)*. 432–439.
- U. Farooq. 2011. Model based test suite minimization using metaheuristics. <http://ro.ecu.edu.au/cgi/viewcontent.cgi?article=1409&context=theses>.
- A. Feliachi and H. L. Guen. 2010. Generating transition probabilities for automatic model-based test generation. In *Proceedings of the 3rd International Conference on Software Testing, Verification, and Validation (ICST'10)*. 99–102.
- P. G. Frankl, G. Rothermel, K. Sayre, and F. I. Vokolos. 2003. An empirical comparison of two safe regression test selection techniques. In *Proceedings of the International Symposium on Empirical Software Engineering (ISESE'03)*. IEEE Computer Society, 195–204.
- F. Galon. 1889. *Natural Inheritance*. MacMillan.
- K. Hamamoto, K. Morooka, and H. Nagahashi. 2004. Motion recognition by combining HMM and reinforcement learning. In *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics (ICSMC'04)*. 259–264.
- J. Hao and E. Mendes. 2006. Usage-based statistical testing of Web applications. In *Proceedings of the 6th International Conference on Web Engineering (ICWE'06)*. ACM Press, New York, 17–24.
- H. Hemmati. 2011. Similarity-based test case selection: Toward scalable and practical model-based testing. <https://www.simula.no/publications/similarity-based-test-case-selection-toward-scalable-and-practical-model-based-testing>.
- H. Hemmati, A. Arcuri, and L. Briand. 2013. Achieving scalable model-based testing through test case diversity. *ACM Trans. Softw. Engin. Methodol.* 22, 1, 1–42.

- M. A. T. Ho, Y. Yamada, and Y. Umetani. 2002. An HMM-based temporal difference learning with model-updating capability for visual tracking of human communicational behaviors. In *Proceedings of the 5th IEEE International Conference on Automatic Face Gesture Recognition (FGR'02)*. 170–175.
- H. Hsu and P. A. Lachenbruch. 2008. Paired t-test. In *Encyclopedia of Clinical Trials*. Wiley, 1–3.
- C.-Y. Huang, J.-R. Chang, and Y.-H. Chang. 2010. Design and analysis of GUI test-case prioritization using weight-based methods. *J. Syst. Softw.* 83, 4, 646–659.
- R. Ihaka and R. Gentleman. 1996. R: A language for data analysis and graphics. *J. Comput. Graph. Statist.* 5, 3, 299–314.
- T. P. Jaakkola, S. P. Singh, and M. I. Jordan. 1995. Reinforcement learning algorithm for partially observable Markov decision problems. <http://papers.nips.cc/paper/951-reinforcement-learning-algorithm-for-partially-observable-markov-decision-problems.pdf>.
- D. Jeffrey and N. Gupta. 2007. Improving fault detection capability by selectively retaining test cases during test suite reduction. *IEEE Trans. Softw. Engin.* 33, 2, 108–123.
- J. Kabudian, M. R. Meybodi, and M. M. Homayounpour. 2004. Applying continuous action reinforcement learning automata (CARLA) to global training of hidden Markov models. In *Proceedings of the International Conference on Information Technology: Coding and Computing (ITCC'04)*. 638–642.
- K. Lee, H. Hon, M. Hwang, and X. Huang. 1990. Speech recognition using hidden Markov model: A CMU perspective. *Speech Comm.* 9, 5–6, 497–508.
- B. Legeard and M. Utting. 2006. *Practical Model-Based Testing: A Tools Approach*. Morgan Kaufmann, San Francisco.
- L. J. Lin. 1992. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Mach. Learn.* 8, 3–4, 293–321.
- F. Lindlar, A. Windisch, and J. Wegener. 2010. Integrating model-based testing with evolutionary functional testing. In *Proceedings of the 3rd International Conference on Software Testing, Verification, and Validation Workshops (ICSTW'10)*. 163–172.
- L. Mariani, M. Pezz, O. Riganelli, and M. Santoro. 2011. AutoBlackTest: A tool for automatic black-box testing. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE'11)*. 1013–1015.
- L. Mariani, M. Pezz, O. Riganelli, and M. Santoro. 2012. AutoBlackTest: Automatic black-box testing of interactive applications. In *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST'12)*. 81–90.
- A. K. McCallum. 1995. Reinforcement learning with selective perception and hidden state. Doctoral dissertation, University of Rochester, NY. <http://web.media.mit.edu/~tristan/Courses/MAS.945/Papers/Contextual/McCallum.Thesis.pdf>.
- M. McPartland and M. Gallagher. 2011. Reinforcement learning in first person shooter games. *IEEE Trans. Comput. Intell. AI Games* 3, 1, 43–56.
- H. Mei, D. Hao, L. Zhang, J. Zhou, and G. Rothermel. 2012. A static approach to prioritizing J-unit test cases. *IEEE Trans. Softw. Engin.* 38, 6, 1258–1275.
- A. M. Memon. 2007. An event-flow model of GUI-based applications for testing. *Softw. Test. Verif. Reliab.* 17, 3, 137–157.
- A. M. Memon and D. R. Hackner. 2008. Test case generator for GUITAR. In *Proceedings of the Companion to the International Conference of Software Engineering (ICSE'08)*. 959–960.
- S. Mirarab and L. Tahvildari. 2007. A prioritization approach for software. In *Proceedings of the 10th International Conference on Fundamental Approaches to Software Engineering (FASE'07)*. 276–290.
- S. Mirarab and L. Tahvildari. 2008. An empirical study on Bayesian network-based approach for test case prioritization. In *Proceedings of the 1st International Conference on Software Testing, Verification, and Validation (ICST'08)*. 278–287.
- D. Nardo, N. Alshahwan, and L. Briand. 2013. Coverage-based test case prioritisation: An industrial case study. In *Proceeding of the 6th International Conference on Software Testing, Verification, and Validation (ICST'13)*. 302–311.
- S. Parsa and A. Khalilian. 2010. On the optimization approach towards test suite minimization approach. *Int. J. Softw. Engin. Appl.* 4, 1, 15–28.
- L. R. Rabiner and B. H. Juang. 1986. An introduction to hidden Markov models. *IEEE ASSP Mag.* 3, 1, 4–16.
- H. Reza, S. Endapally, and E. Grant. 2007. A model-based approach for testing GUI using hierarchical predicate transition nets. In *Proceedings of the 4th International Conference on Information Technology (ITNG'07)*. 366–370.
- J. A. Rice. 1994. *Mathematical Statistics and Data Analysis*. Duxbury Press.

- M. Robinson and P. A. Vorobiev. 1999. *Swing: A Fast-Paced Guide with Production-Quality Code Examples*. Manning.
- G. Rothermel and D. Hall. 1997. A safe, efficient regression test selection technique. *ACM Trans. Softw. Engin. Methodol.* 6, 2, 173–210.
- G. Rothermel and M. J. Harrold. 1998. Empirical studies of a safe regression test selection technique. *IEEE Trans. Softw. Engin.* 24, 6, 108–123.
- G. Rothermel, M. J. Harrold, J. Ostrin, and C. Hong. 1998. An empirical study of the effects of minimization on the fault detection capabilities of test suites. In *Proceedings of the International Conference on Software Maintenance (ICSM'98)*. 34–43.
- G. Rothermel, R. Untch, and M. J. Harrold. 2001. Prioritizing test cases for regression testing. *IEEE Trans. Softw. Engin.* 27, 10, 929–948.
- G. S. Semmel and D. G. Linton. 1998. Determining optimal testing times for Markov chain usage models. In *Proceedings of the IEEE SECON Conference (Southeastcon'98)*. 1–4.
- R. S. Sutton and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, Massachusetts.
- C. Szepesvári. 2010. *Algorithms for Reinforcement Learning*. Morgan and Claypool.
- S. W. Thomas, H. Hemmati, A. E. Hassan, and D. Blostein. 2012. Static test case prioritization using topic models. *Empir. Softw. Engin.* 19, 1, 182–212.
- O. Tramasco and S. Bauer. 2013. Package 'RHmm'. <http://www2.uaem.mx/r-mirror/web/packages/RHmm/RHmm.pdf>.
- M. A. Walker. 2000. An application of reinforcement learning to dialogue strategy selection in a spoken dialogue system for email. *J. Artif. Intell. Res.* 12, 387–416.
- C. J. Watkin. 1989. Learning from delayed rewards. 1–14. https://www.cs.rhul.ac.uk/home/chrisw/new_thesis.pdf.
- A. L. White and J. A. Sjorgen. 1988. Markov chains for testing redundant software. In *Proceedings of the Reliability and Maintainability Symposium (RAMS'88)*. 426–433.
- J. A. Whittaker and M. G. Thomason. 1994. A Markov chain model for statistical software testing. *IEEE Trans. Softw. Engin.* 20, 10, 812–824.
- M. Wiering and M. Van Otterlo. 2012. *Reinforcement Learning (State of the Art)*. Springer.
- J. D. Williams and S. Young. 2007. Partially observable Markov decision processes for spoken dialog systems. *Comput. Speech Lang.* 21, 2, 393–422.
- W. E. Wong, J. R. Horgan, S. London, and H. Agrawal. 1997. A study of effective regression testing in practice. In *Proceedings of the 8th IEEE International Symposium on Software Reliability Engineering (ISSRE'97)*. 264–274.
- M. M. Yin and J. T. L. Wang. 2001. Effective hidden Markov models for detecting splicing junction sites in DNA sequences. *Inf. Sci.* 139, 1–2, 139–163.
- S. Yoo and M. Harman. 2007. Regression testing minimisation, selection and prioritisation: A survey. *Softw. Test. Verif. Reliab.* 7, 1–60.
- Y. T. Yu and M. F. Lau. 2012. Fault-based test suite prioritization for specification-based testing. *Inf. Softw. Technol.* 54, 2, 179–202.
- L. Zhang, S.-S. Hou, C. Guo, T. Xie, and H. Mei. 2009a. Time-aware test-case prioritization using integer linear programming. In *Proceedings of the 18th International Symposium on Software Testing and Analysis (ISSTA'09)*. ACM Press, New York, 401–419.
- Z. Zhang, W. K. Chan, and T. H. Tse. 2009b. Adaptive random test case prioritization. In *Proceedings of the 24th IEEE/ACM International Conference on Automated Software Engineering (ASE'09)*.
- F. Zhen And P. Chenglian. 2004. A system test methodology based on the Markov chain usage model. In *Proceedings of the 8th International Conference on Computer Supported Cooperative Work in Design (CACWD'04)*. 160–165.

Received May 2014; revised November 2014, March 2015, May 2015; accepted June 2015